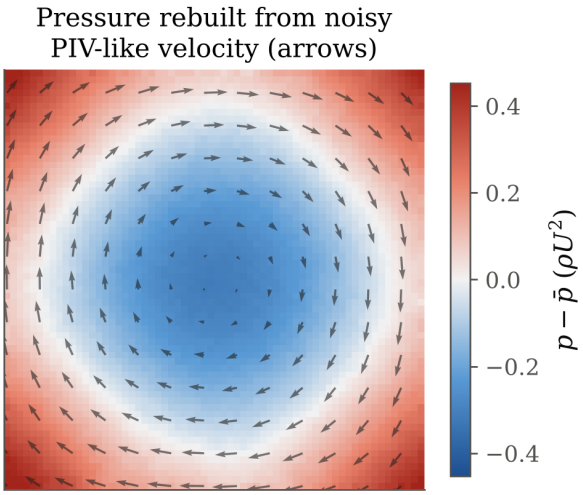
ME 415 · Computational Fluid Dynamics and Heat Transfer · IIT Bombay

Pressure from PIV

Algorithms and Comparison with Simulations

The complete explainer: from first principles to every result



Anany Bajpai (24B2135) · Arnav Oza (24B2217)

Instructor: Prof. Anjishnu Choudhury

Term project 6 · Autumn 2026 · Document version of 31 October 2026

How to read this document

This document explains the whole project from the ground up. It assumes you know undergraduate fluid mechanics and basic numerical methods, and nothing about PIV or pressure reconstruction. It goes all the way down to individual stencils, matrix rows and the reasons behind every design decision, so you can defend any line of the code in a viva.

- **Part I (Chapters 1–3)** builds the physics: what PIV measures, why pressure is not measured, and how the Navier–Stokes equations let us recover it.
- **Part II (Chapters 4–7)** builds the numerics: finite differences and how they amplify noise, the discrete Poisson problem and its singularity, the three integration algorithms, and the iterative solvers.
- **Part III (Chapters 8–9)** sets up the virtual experiment: exact flows, the synthetic PIV error model, metrics, and the code that runs it all.
- **Part IV (Chapters 10–11)** presents every result with the reason it comes out the way it does.
- **Part V (Chapters 12–14)** covers the OpenFOAM cylinder-wake simulation, the comparison with published studies, and the conclusions.
- **Appendices** collect viva questions with short answers, a file map, the run commands and the notation.

Three kinds of boxes recur:

Key idea

A result or principle worth remembering.

Pitfall

Something that goes wrong if done naively, and what the project does instead.

Viva questions

Questions an examiner is likely to ask, with short answers.

Sources of the numbers. Every analytic result comes from the project's own CSV files (`results/*.csv`, 10 noise seeds, deterministic). The cylinder-wake results in Chapter 12 come from the OpenFOAM runs (three meshes) and their post-processing scripts.

Contents

How to read this document i

I Physics: what we want and why it is hard 1

1 The project in one picture 2
- 1.1 The question . . . 2
- 1.2 The answer in three steps . . . 2
- 1.3 What was built . . . 3

2 Particle Image Velocimetry 4
- 2.1 How PIV works . . . 4
- 2.2 What PIV gets wrong . . . 4

3 From velocity to pressure: the governing equations 6
- 3.1 Pressure gradient from Navier–Stokes . . . 6
- 3.2 The material derivative: two ways to estimate it . . . 6
  - 3.2.1 Eulerian (fixed point) . . . 6
  - 3.2.2 Lagrangian (follow the particle) . . . 6
- 3.3 The pressure-Poisson equation . . . 7
  - 3.3.1 Compatibility . . . 7
  - 3.3.2 Poisson as a least-squares projection . . . 7
  - 3.3.3 An alternative source term (2-D incompressible identity) . . . 8
- 3.4 Bernoulli boundary values . . . 8

II Numerics: turning the equations into code 9

4 Finite differences and noise amplification 10
- 4.1 Stencils used everywhere . . . 10
- 4.2 How differentiation amplifies noise . . . 10

5 The discrete pressure-Poisson problem 12
- 5.1 The 5-point Laplacian . . . 12
- 5.2 Neumann boundaries by ghost-point elimination . . . 12
- 5.3 The all-Neumann system is singular . . . 12

6 Direct and omni-directional integration 14

6.1 Direct spatial integration . . . 14
6.2 Omni-directional integration . . . 14

7 Iterative solvers, norms and convergence 16
7.1 Norms of a residual . . . 16
7.2 Jacobi is pseudo-time marching . . . 16
7.3 Gauss–Seidel and SOR . . . 17
7.4 Conjugate gradient . . . 17
7.5 Residual vs change: when is a solution converged? . . . 17

III The virtual experiment 19

8 Test flows, synthetic PIV and error metrics 20
8.1 Taylor–Green vortex . . . 20
8.2 Kovasznay flow . . . 21
8.3 The synthetic PIV model . . . 21
8.4 Denoising . . . 21
8.5 Error metrics . . . 22
8.6 Experiment design . . . 22

9 The code 23
9.1 Architecture . . . 23
9.2 Pseudo-code . . . 23
9.3 Inside the Poisson solve . . . 23
9.4 Tests: how we know the code is right . . . 24

IV Results and what they mean 25

10 Reconstruction results 26
10.1 Verification: grid convergence on clean data . . . 26
10.2 Noise vs resolution: an optimum spacing . . . 27
10.3 Noise sensitivity and choice of method . . . 28
10.4 Boundary conditions for the Poisson solver . . . 29
10.5 Time step: Eulerian vs Lagrangian . . . 30
10.6 Interrogation window and denoising . . . 31

11 Solver and convergence results 32
11.1 Residual histories . . . 32
11.2 Iterations vs grid size and SOR stability . . . 33

11.3 Residual vs change . . . 34
11.4 Norms across grids . . . 34
11.5 First- vs second-order schemes . . . 35

V Simulation, literature and conclusions 36

12 The CFD case: a cylinder wake at Re = 100 37
12.1 Why this flow . . . 37
12.2 Domain, mesh and boundary conditions . . . 37
12.3 Solver and numerics . . . 38
12.4 Outputs and how they are processed . . . 38
12.4.1 Strouhal number, drag and lift . . . 39
12.4.2 Is the wake fully developed? Periodicity, not steady state . . . 39
12.4.3 Grid independence: Richardson extrapolation and GCI . . . 39
12.5 Results . . . 39
12.5.1 The flow field . . . 40
12.5.2 Grid independence . . . 40
12.5.3 Residuals and periodicity . . . 42
12.5.4 Pressure reconstructed from the CFD velocity . . . 42
12.5.5 Time step, scheme and noise on the wake . . . 44

13 Comparison with published studies 45

14 Conclusions 46
14.1 Key findings . . . 46
14.2 Objectives . . . 46
14.3 Recommended practice . . . 46
14.4 Limitations and future work . . . 47

A Viva preparation: questions and short answers 48

B File map and how to run 51
B.1 Files . . . 51
B.2 Commands . . . 51

C Notation 53

References 54

Part I

Physics: what we want and why it is hard

Chapter 1

The project in one picture

1.1 The question

Pressure is one of the most useful quantities in fluid mechanics. It sets the lift and drag on a body, drives cavitation, and is the source of flow noise. Yet it is surprisingly hard to measure over a whole field:

- **Pressure taps** and transducers give pressure only at a few points, and only on walls.
- **Probes** placed in the flow disturb the very flow they measure.
- **Pressure-sensitive paint** works only on surfaces.

Particle Image Velocimetry (PIV), by contrast, measures the *velocity* over a whole plane without touching the flow. The question of this project is:

The question

Given only PIV-type velocity fields $\mathbf{u}(x, y, t)$ (noisy, smoothed, on a finite grid), how accurately can we reconstruct the pressure field $p(x, y, t)$, which algorithm should we use, and what controls the error?

1.2 The answer in three steps

The momentum equation links pressure to velocity. Rearranged, it gives the pressure *gradient* as a function of velocity only:

$$\nabla p = \mathbf{G} \equiv -\rho \frac{D\mathbf{u}}{Dt} + \mu \nabla^2 \mathbf{u}. \tag{1.1}$$

So the recipe is:

1. **Differentiate** the measured velocity (in space and time) to get $\mathbf{G}$.
2. **Integrate** $\mathbf{G}$ to get p (up to an additive constant).
3. **Compare** with a known true pressure to measure the error.

Step 1 is where the difficulty lives: differentiation amplifies measurement noise. Step 2 decides whether that noise is averaged out or accumulated. Step 3 needs data where the true pressure is known, which is why the project uses exact Navier–Stokes solutions and a CFD simulation as "virtual experiments".

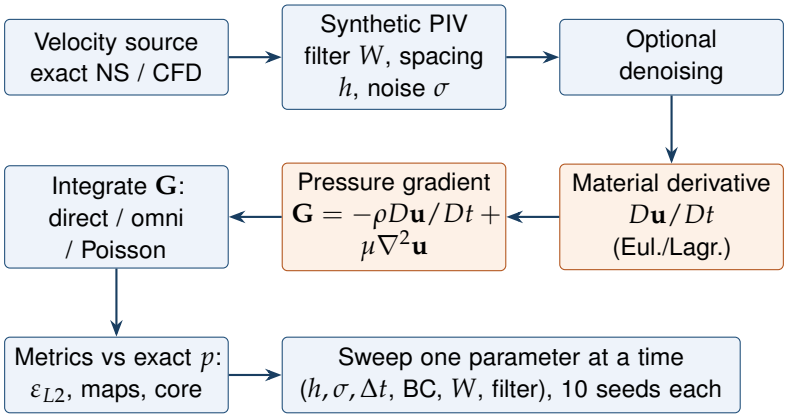


Figure 1.1: The whole project as one pipeline. Orange boxes are where noise is amplified (differentiation); the integration step decides whether it is averaged out.

1.3 What was built

- A Python package (about 7 500 lines, NumPy/SciPy/Matplotlib) with three reconstruction algorithms: **direct integration**, **omni-directional integration** and a **pressure-Poisson solver**, plus five Poisson solvers (sparse direct, Jacobi, Gauss–Seidel, SOR, conjugate gradient).
- A **synthetic PIV model** that degrades exact velocity fields like a real PIV system: window averaging, finite spacing, random noise.
- **Validation flows**: the Taylor–Green vortex (stationary and convected) and Kovasznay flow (Re 20 and 40), all exact solutions of Navier–Stokes.
- A **CFD case**: a 2-D laminar cylinder wake at Re = 100 in OpenFOAM, on three meshes, sampled in a PIV-like window.
- **Studies** of grid spacing, noise, time step, boundary conditions, integration method, window size and denoising; a convergence and error-analysis study (norms, residuals, solver stability, scheme order).
- **87 unit tests**, all passing, which check every stencil, solver and parser against exact answers.

Chapter 2

Particle Image Velocimetry

2.1 How PIV works

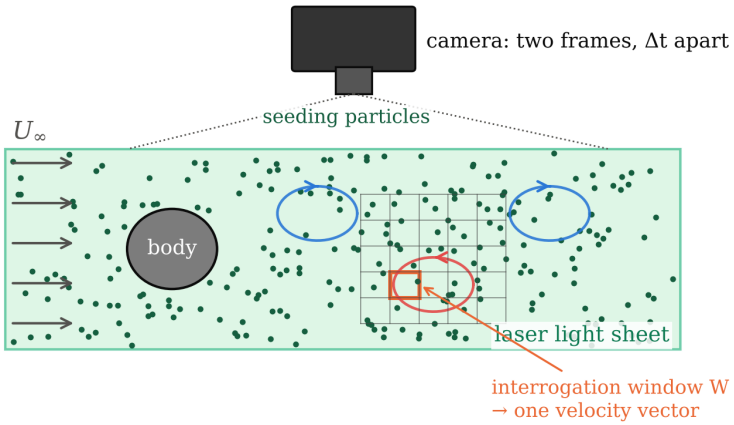


Figure 2.1: Planar PIV. A laser sheet lights seeding particles; a camera records two frames Δt apart; each interrogation window yields one velocity vector.

1. The flow is seeded with small tracer particles that follow it faithfully.
2. A thin laser sheet illuminates a plane of the flow.
3. A camera takes two images a short time Δt apart.
4. Each image is cut into small **interrogation windows** of size W. Cross-correlating a window in frame 1 with the same window in frame 2 gives the most likely particle displacement $\Delta\mathbf{x}$ in that window.
5. The velocity of the window is $\mathbf{u} = \Delta\mathbf{x}/\Delta t$. Repeating over all windows gives a vector field on a grid of spacing h (often $h = W/2$ with 50 % overlap).
6. **Time-resolved PIV** repeats this at high frame rate, giving a sequence of fields $\mathbf{u}^{n-1}, \mathbf{u}^{n}, \mathbf{u}^{n+1}, \ldots$ This is what the time derivative in $D\mathbf{u}/Dt$ needs.

2.2 What PIV gets wrong

Every pressure error in this project traces back to three properties of PIV data. The synthetic model in Chapter 8 reproduces exactly these three.

1. Spatial averaging (window W)
A vector is an average over the window, so it acts like a low-pass filter. Peaks are flattened and sharp gradients are smeared. For a box average of width W the transfer function is $\sin(kW/2)/(kW/2) \approx 1 - (kW)^2/24$, i.e. the measured velocity is biased by about $-(W^2/24)\nabla^2\mathbf{u}$.

2. Finite spacing (h)
Derivatives must be estimated by finite differences on a grid of spacing h; their truncation error falls as h^2 but noise amplification grows as $1/h$ or $1/h^2$.

3. Random error (σ)
Correlation peaks are located with an uncertainty of order 0.1 pixel, giving a random velocity error typically 0.5–2 % of the flow speed. We model it as independent Gaussian noise of standard deviation σU.

Real PIV adds further errors: out-of-plane motion (planar PIV misses w and $\partial/\partial z$), particle lag, peak locking, and outliers. These are discussed as limitations in Chapter 14.

Chapter 3

From velocity to pressure: the governing equations

3.1 Pressure gradient from Navier–Stokes

For an incompressible Newtonian fluid with constant density ρ and viscosity μ:

$$\nabla \cdot \mathbf{u} = 0, \qquad \rho\left(\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u}\right) = -\nabla p + \mu\nabla^2\mathbf{u}. \tag{3.1}$$

Solve the momentum equation for ∇p:

$$\boxed{\nabla p = \mathbf{G} \equiv -\rho\frac{D\mathbf{u}}{Dt} + \mu\nabla^2\mathbf{u}, \qquad \frac{D\mathbf{u}}{Dt} = \frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u}\cdot\nabla)\mathbf{u}} \tag{3.2}$$

In two dimensions, component by component (this is literally what `core/material_derivative.py` computes):

$$G_x = -\rho\,(u_t + u\,u_x + v\,u_y) + \mu\,(u_{xx} + u_{yy}), \tag{3.3}$$

$$G_y = -\rho\,(v_t + u\,v_x + v\,v_y) + \mu\,(v_{xx} + v_{yy}). \tag{3.4}$$

- $D\mathbf{u}/Dt$ is the **material (particle) acceleration**. It dominates at moderate and high Reynolds number.
- $\mu\nabla^2\mathbf{u}$ is the **viscous term**. It matters at low Reynolds number: for Kovasznay flow at Re = 20 or 40, dropping it leaves an $O(1)$ error that never converges (a unit test checks this).
- In the non-dimensional CFD setup $\rho = 1$, so p is the *kinematic* pressure p/ρ, exactly what OpenFOAM's incompressible solvers use.

Why this is an inverse problem

Every term of $\mathbf{G}$ is a *derivative* of measured data. So pressure from PIV is the inverse of differentiation: the data are differentiated (which amplifies noise) and then integrated (which can smooth it again). Everything else in the project is about doing these two operations well.

3.2 The material derivative: two ways to estimate it

3.2.1 Eulerian (fixed point)

At a fixed grid node, with three snapshots $n-1, n, n+1$:

$$\left(\frac{D\mathbf{u}}{Dt}\right)_E \approx \frac{\mathbf{u}^{n+1} - \mathbf{u}^{n-1}}{2\Delta t} + (\mathbf{u}^n\cdot\nabla)\mathbf{u}^n. \tag{3.5}$$

The time difference has truncation error $-(\Delta t^2/6)\,\partial^3\mathbf{u}/\partial t^3$: second order.

3.2.2 Lagrangian (follow the particle)

Instead of sitting at a point, follow a fluid particle along a straight path through the node with the local velocity $\mathbf{a} = \mathbf{u}(\mathbf{x}, t^n)$:

$$\left(\frac{D\mathbf{u}}{Dt}\right)_L \approx \frac{\mathbf{u}(\mathbf{x} + \mathbf{a}\Delta t,\, t^{n+1}) - \mathbf{u}(\mathbf{x} - \mathbf{a}\Delta t,\, t^{n-1})}{2\Delta t}. \tag{3.6}$$

The velocities at the off-grid points come from cubic interpolation.

Accuracy. Let $L = \partial/\partial t + \mathbf{a}\cdot\nabla$ with $\mathbf{a}$ frozen. A shift in space and time is the exponential of L: $\mathbf{u}(\mathbf{x} \pm \mathbf{a}\Delta t, t \pm \Delta t) = e^{\pm \Delta t L}\mathbf{u}$. Then

$$\left(\frac{D\mathbf{u}}{Dt}\right)_L = \frac{\sinh(\Delta t L)}{\Delta t}\mathbf{u} = L\mathbf{u} + \frac{\Delta t^2}{6}L^3\mathbf{u} + O(\Delta t^4), \tag{3.7}$$

and $L\mathbf{u}$ is exactly $D\mathbf{u}/Dt$. So the Lagrangian scheme is also second order in Δt; what differs is the *size* of the error constant.

When Lagrangian wins

Take a flow pattern $\mathbf{w}$ carried along at speed $\mathbf{c}$: $\mathbf{u}(\mathbf{x}, t) = \mathbf{c} + \mathbf{w}(\mathbf{x} - \mathbf{c}t)$.

- Eulerian error $\propto \partial^3\mathbf{u}/\partial t^3 = -(\mathbf{c}\cdot\nabla)^3\mathbf{w}$, which scales like c^3.
- Lagrangian error $\propto L^3\mathbf{u} = (\mathbf{w}\cdot\nabla)^3\mathbf{w}$, which scales like w^3.

In a wake the vortices are *carried* faster ($c \approx U$) than they *spin* ($w < c$), so the Lagrangian estimate is more accurate at large Δt. For a stationary vortex ($c = 0$) the Eulerian time error is almost zero and the Lagrangian one is larger. The project verifies both cases (Section 10.5).

SciPy cubic interpolation

Since SciPy 1.13, `RegularGridInterpolator(method="cubic")` computes spline coefficients with an iterative solver (atol 10^{-6}). The interpolation error then stalls near 10^{-5} whatever the grid, and the Lagrangian error *grows* under refinement. The code requests the direct solver (`solver=spsolve`), which restores fourth-order interpolation. Where the tracked point leaves the window, the code falls back to the Eulerian value and reports the fraction of such nodes.

3.3 The pressure-Poisson equation

Taking the divergence of $\nabla p = \mathbf{G}$ gives an elliptic problem with Neumann boundary conditions:

$$\boxed{\nabla^2 p = \nabla\cdot\mathbf{G} \ \text{ in } \Omega, \qquad \frac{\partial p}{\partial n} = \mathbf{n}\cdot\mathbf{G} \ \text{ on } \partial\Omega} \tag{3.8}$$

3.3.1 Compatibility

Integrate the Poisson equation over the domain and use Gauss's theorem: $\iint_\Omega \nabla^2 p\, dA = \oint_{\partial\Omega} \partial p/\partial n\, ds$. A solution exists only if

$$R \equiv \iint_\Omega \nabla\cdot\mathbf{G}\, dA - \oint_{\partial\Omega} \mathbf{n}\cdot\mathbf{G}\, ds = 0. \tag{3.9}$$

For an exact gradient field $R = 0$ automatically. For discretised or noisy $\mathbf{G}$ it is not, and the code reports R (`compatibility_residual`). Even when it holds, the solution is unique only up to a constant: adding any constant to p changes nothing.

3.3.2 Poisson as a least-squares projection

Measured $\mathbf{G}$ is not exactly a gradient: it has a non-zero curl, which can only be error (the curl of a true gradient is zero). What is the "best" pressure? Minimise the mismatch

$$J[p] = \iint_\Omega |\nabla p - \mathbf{G}|^2\, dA. \tag{3.10}$$

The calculus of variations gives the Euler–Lagrange equation $\nabla^2 p = \nabla \cdot \mathbf{G}$ in Ω and the natural boundary condition $\partial p/\partial n = \mathbf{n} \cdot \mathbf{G}$ on $\partial\Omega$: exactly Eq. (3.8).

Why Poisson is robust

The Poisson problem *projects* the measured $\mathbf{G}$ onto the space of gradient fields. It keeps the curl-free part and discards the curl, which is pure error. That is why Poisson reconstruction is path-independent and filters noise, and why the source term and the boundary data must come from the *same* $\mathbf{G}$ (Section 10.3 shows what happens otherwise).

3.3.3 An alternative source term (2-D incompressible identity)

Take the divergence of $\mathbf{G}$ term by term. $\nabla \cdot (\partial \mathbf{u}/\partial t) = \partial(\nabla \cdot \mathbf{u})/\partial t = 0$ and $\nabla \cdot \nabla^2 \mathbf{u} = \nabla^2(\nabla \cdot \mathbf{u}) = 0$. Only the convective term survives:

$$\nabla \cdot ((\mathbf{u} \cdot \nabla)\mathbf{u}) = (\partial_i u_j)(\partial_j u_i) + u_j \partial_j (\partial_i u_i) = u_x^2 + 2u_y v_x + v_y^2, \tag{3.11}$$

so

$$\nabla^2 p = -\rho \left(u_x^2 + 2u_y v_x + v_y^2 \right). \tag{3.12}$$

This form needs no time derivative and no viscous term, which looks attractive. But it assumes $\nabla \cdot \mathbf{u} = 0$ exactly (noisy PIV data are not solenoidal), it multiplies noisy derivatives together (a bias of order σ^2/h^2), and its boundary data still come from $\mathbf{G}$. Section 10.3 shows that it is 5–8× worse on noisy data.

Check on Taylor–Green: $u_x = -\sin x \sin y\, F$, $v_y = \sin x \sin y\, F$, $u_y = \cos x \cos y\, F$, $v_x = -\cos x \cos y\, F$, so $-\rho(2\sin^2 x \sin^2 y - 2\cos^2 x \cos^2 y)F^2 = \rho(\cos 2x + \cos 2y)F^2 = \nabla^2 p$. ✓

3.4 Bernoulli boundary values

For **steady, irrotational** flow, $p + \frac{1}{2}\rho|\mathbf{u}|^2$ is constant along the whole field, so a free-stream edge could be given the Dirichlet value

$$p = p_\infty + \tfrac{1}{2}\rho\,(U_\infty^2 - |\mathbf{u}|^2). \tag{3.13}$$

For unsteady irrotational flow the correct form is $p + \rho\,\partial\phi/\partial t + \frac{1}{2}\rho|\mathbf{u}|^2 = \text{const}$, and in rotational flow there is no such relation at all. Both validation flows carry vorticity everywhere, so Bernoulli edges *must* fail there; the project tests this deliberately (Section 10.4).

Viva questions

1. *Why can't PIV measure pressure directly?* It measures particle displacement, i.e. velocity. Pressure enters only through the momentum equation.
2. *Why is the result only defined up to a constant?* Only ∇p is known; a constant has zero gradient. We compare $p - \bar{p}$ with $p_{ref} - \bar{p}_{ref}$.
3. *When can you neglect the viscous term?* At high Reynolds number away from walls; never in Kovasznay flow at Re 20–40.
4. *What does the compatibility condition mean physically?* The total "source" inside must equal the net flux through the boundary; a non-gradient $\mathbf{G}$ violates it.
5. *Why does Poisson filter noise?* It is the least-squares fit of a gradient field to $\mathbf{G}$; the curl part (pure error) is discarded.

Part II

Numerics: turning the equations into code

Chapter 4

Finite differences and noise amplification

4.1 Stencils used everywhere

All spatial derivatives use second-order finite differences on the uniform PIV grid (spacing h), including at the edges.

Quantity	Where	Stencil	Truncation error
f'	interior	$(f_{i+1} - f_{i-1})/2h$	$-\frac{h^2}{6} f'''$
f'	left edge	$(-3f_0 + 4f_1 - f_2)/2h$	$+\frac{h^2}{3} f'''$
f''	interior	$(f_{i+1} - 2f_i + f_{i-1})/h^2$	$-\frac{h^2}{12} f''''$
f''	left edge	$(2f_0 - 5f_1 + 4f_2 - f_3)/h^2$	$+\frac{11h^2}{12} f''''$
$\partial_t f$	middle snapshot	$(f^{n+1} - f^{n-1})/2\Delta t$	$-\frac{\Delta t^2}{6} f_{ttt}$

Table 4.1: The finite-difference stencils (`core/derivatives.py`). The right edge mirrors the left.

Deriving the one-sided first derivative. Expand about x_0: $f_1 = f_0 + hf' + \frac{h^2}{2}f'' + \frac{h^3}{6}f'''$ and $f_2 = f_0 + 2hf' + 2h^2 f'' + \frac{4h^3}{3}f'''$. Look for $(af_0 + bf_1 + cf_2)/h$ whose f_0 and f'' coefficients vanish and whose f' coefficient is 1:

$$a + b + c = 0, \qquad b + 2c = 1, \qquad \tfrac{b}{2} + 2c = 0 \;\Rightarrow\; a = -\tfrac{3}{2},\; b = 2,\; c = -\tfrac{1}{2}.$$

The same procedure with four points (cancel f_0, f', f''', keep f'') gives $(2, -5, 4, -1)/h^2$.

Boundary stencils must be second order too

The Neumann data $\mathbf{n} \cdot \mathbf{G}$ and the source term on the boundary rows are computed with these edge stencils. A first-order stencil there would pull the *global* order of the whole reconstruction down to 1. (Chapter 11 shows this deliberately with `order=1`.)

4.2 How differentiation amplifies noise

Suppose every velocity sample carries independent noise of standard deviation $s = \sigma U$. Linear combinations of independent errors add in variance:

$$\text{time difference: } \operatorname{std}\left[\frac{\epsilon^{n+1} - \epsilon^{n-1}}{2\Delta t}\right] = \frac{\sqrt{2}\,s}{2\Delta t} = \frac{s}{\sqrt{2}\,\Delta t}, \tag{4.1}$$

$$\text{first derivative: } \frac{s}{\sqrt{2}\,h}, \tag{4.2}$$

$$\text{5-point Laplacian: } \frac{\sqrt{1^2 + 1^2 + 1^2 + 1^2 + 4^2}\,s}{h^2} = \frac{\sqrt{20}\,s}{h^2}. \tag{4.3}$$

Putting these into $\mathbf{G}$:

$$\operatorname{noise}(\mathbf{G}) \sim \underbrace{\rho \frac{s}{\sqrt{2}\Delta t}}_{\partial \mathbf{u}/\partial t} + \underbrace{\rho U \frac{s}{\sqrt{2}h}}_{\text{convective}} + \underbrace{\mu \frac{\sqrt{20}\,s}{h^2}}_{\text{viscous}}. \tag{4.4}$$

The central tension of the whole project

Truncation error falls as h^2 and Δt^2, but noise amplification grows as $1/h$, $1/h^2$ and $1/\Delta t$. Refining the measurement therefore helps only up to a point. Below an optimum spacing the error *rises* again. Minimising $E(h) \approx Ch^2 + K\sigma/h^q$ gives $h_{opt} \propto \sigma^{1/(2+q)}$: noisier data need a coarser grid.

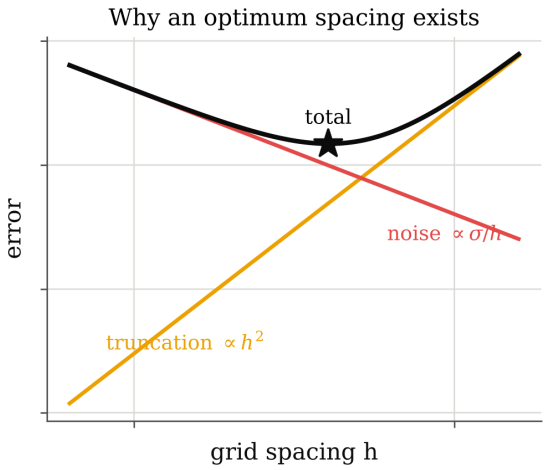


Figure 4.1: Error budget (schematic): truncation error falls with h, noise amplification rises, and the total has a minimum.

Chapter 5

The discrete pressure-Poisson problem

5.1 The 5-point Laplacian

Unknowns $p_{i,j}$ sit at the PIV nodes, numbered $k = j\,n_x + i$. In the interior,

$$\frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{\Delta x^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{\Delta y^2} = f_{i,j} = (\nabla \cdot \mathbf{G})_{i,j}, \tag{5.1}$$

with truncation error $-\frac{\Delta x^2}{12} p_{xxxx} - \frac{\Delta y^2}{12} p_{yyyy}$. The matrix is assembled with Kronecker products, $A = I_y \otimes D_{xx} + D_{yy} \otimes I_x$, and stored as a sparse CSR matrix.

5.2 Neumann boundaries by ghost-point elimination

At the left edge ($i = 0$) invent a ghost node $i = -1$ outside the domain. Discretise the Neumann condition $\partial p/\partial x = G_x$ centrally:

$$\frac{p_1 - p_{-1}}{2\Delta x} = G_{x,0} \quad \Rightarrow \quad p_{-1} = p_1 - 2\Delta x\, G_{x,0}. \tag{5.2}$$

Substitute into the Laplacian row at $i = 0$ and move the known part to the right-hand side:

$$\frac{2p_1 - 2p_0}{\Delta x^2} + [\,y\text{-part}\,] = f_0 + \frac{2G_{x,0}}{\Delta x}. \tag{5.3}$$

The other edges follow the same pattern: right edge $b \mathrel{-}= 2G_x/\Delta x$, bottom $b \mathrel{+}= 2G_y/\Delta y$, top $b \mathrel{-}= 2G_y/\Delta y$. In 1-D the operator becomes the matrix with rows $[-2,\ 2]$, $[1, -2, 1], \ldots, [2, -2]$, all divided by h^2.

Corners have two ghost neighbours, one in x and one in y. Each is eliminated with *its own* condition ($\partial p/\partial x = G_x$ and $\partial p/\partial y = G_y$), so both are used exactly. Averaging the two conditions (as the original project specification suggested) would throw information away.

Order of accuracy. The boundary rows have local truncation error $O(h)$, since $(2p_1 - 2p_0)/h^2 - 2g/h = p'' + \frac{h}{3}p''' + \ldots$ But the global error is still $O(h^2)$:

- the $O(h)$ error lives on a strip of width $O(h)$ only;
- multiplying edge rows by $\frac{1}{2}$ and corner rows by $\frac{1}{4}$ turns the scheme into a vertex-centred finite-volume scheme on half and quarter cells, whose flux errors are $O(h^2)$.

The tests confirm global slopes of 1.8–2.2.

Symmetric form. With trapezoidal weights w (1 inside, $\frac{1}{2}$ on edges, $\frac{1}{4}$ at corners) and $W = \mathrm{diag}(w)$, the matrix WA is symmetric and $S = -WA$ is symmetric positive semi-definite. Conjugate gradients needs exactly this.

5.3 The all-Neumann system is singular

$A\mathbf{1} = 0$: adding a constant to p changes nothing, so A has a one-dimensional null space and the pressure is fixed only up to a constant. The left null vector is w ($w^T A = 0$), so $Ap = b$ is solvable

only if $w^T b = 0$, and

$$\Delta x\,\Delta y\, w^T b = \text{trapezoidal} \iint f\, dA - \text{trapezoidal} \oint \mathbf{n}\cdot\mathbf{G}\, ds, \tag{5.4}$$

the *discrete compatibility residual*. Two fixes are implemented:

`pin="node"`**: replace one row by** $p_k = 0$
The system becomes solvable, but the discarded row was the only place the incompatibility R could go. R then acts as a point source at the pinned node, producing a logarithmic disturbance $\sim (R/2\pi)\ln r$ around it. Errors are larger: on clean Taylor–Green 5.8×10^{-3} vs 7.2×10^{-4}.

`pin="mean"` **(default): zero-mean augmented system**

$$\begin{bmatrix} A & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix}\begin{bmatrix} p \\ \lambda \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix} \quad\Longleftrightarrow\quad Ap + \lambda\mathbf{1} = b,\ \ \textstyle\sum p = 0. \tag{5.5}$$

Multiplying the first row by w^T gives $\lambda = w^T b / w^T\mathbf{1}$: a *uniform* source correction spreads the incompatibility evenly instead of dumping it at one node. The bordered matrix is non-singular because $\mathbf{1}$ is not in the range of A ($w^T\mathbf{1} \neq 0$).

Dirichlet edges (known pressure, from taps or Bernoulli) replace those rows by $p_k =$ value. Such an edge fixes the constant, so no pin is needed.

Viva questions

1. *Why is the all-Neumann matrix singular?* $A\mathbf{1} = 0$; fix it by pinning a node or adding a zero-mean constraint.
2. *Why is pin-mean better than pin-node?* Pin-node dumps the incompatibility R at one point (a point source); pin-mean spreads it uniformly.
3. *What does λ represent?* The uniform source correction that makes the data compatible.
4. *Why is the global error second order if boundary rows are first order?* The $O(h)$ error acts only on an $O(h)$ strip; equivalently the scaled scheme is a second-order finite-volume scheme.

Chapter 6

Direct and omni-directional integration

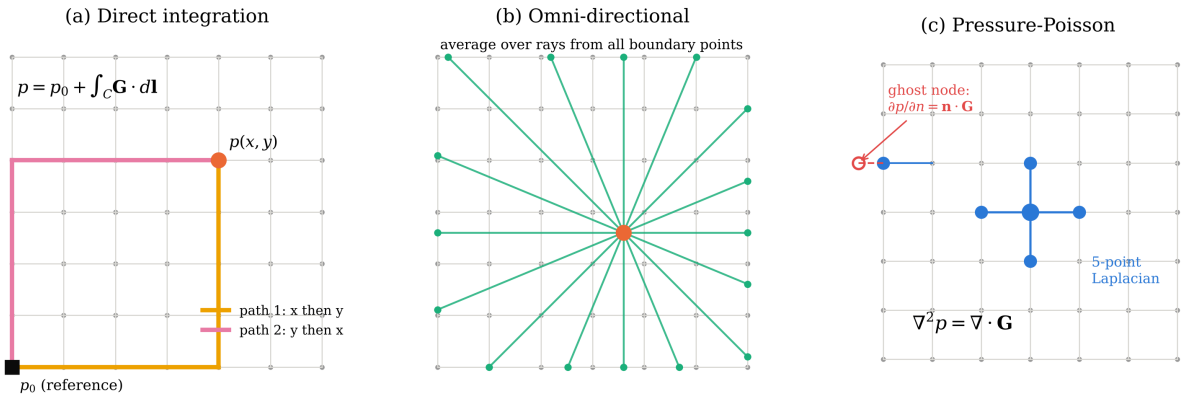


Figure 6.1: The three algorithms on the PIV grid: (a) direct integration along two paths, (b) omni-directional rays from all directions, (c) the Poisson stencil with a ghost node.

6.1 Direct spatial integration

The simplest idea: integrate the gradient along a path from a reference point,

$$p(B) - p(A) = \int_A^B \mathbf{G} \cdot d\mathbf{l}, \tag{6.1}$$

using the cumulative trapezoidal rule (second order).

- **Path 1:** along the first row in x, then up each column in y.
- **Path 2:** along the first column in y, then along each row in x.
- The code reports both and their average.

For an exact gradient both paths agree (Stokes: $\oint \mathbf{G} \cdot d\mathbf{l} = \iint \operatorname{curl} \mathbf{G}\, dA = 0$). Measured $\mathbf{G}$ has a curl, so they differ; the difference measures path dependence.

Why direct integration is so noise-sensitive

Along a path of $N = L/h$ steps the trapezoidal sum adds N independent errors of size $h\,\sigma_G$. They add like a random walk: error $\propto h\,\sigma_G\sqrt{N} = \sigma_G\sqrt{hL}$. With $\sigma_G \sim s/h$ from the derivatives this is $\propto s\sqrt{L/h}$, which *grows* as the grid is refined, and faster still where the viscous noise $\sim s/h^2$ matters. Every error made early on the path is carried to every later point.

6.2 Omni-directional integration

Liu & Katz (2006) proposed averaging line integrals over many paths from all directions, so that path errors cancel. The project implements a *rotating-ray* variant (`core/omni_integration.py`):

1. Directions $\theta_k = 2\pi k/N$, $k = 0, \ldots, N-1$, with $N = 32$.

2. From every node P, trace a straight ray backwards to the boundary point $E_k(P)$ and integrate $\mathbf{G} \cdot \mathbf{e}_k$ from E_k to P (trapezoidal rule, step h, bilinear interpolation of $\mathbf{G}$).
3. **Boundary stage.** Boundary values depend on each other through the rays: $p_i = \text{mean}_k \left[\sum_m A_{im} p_m + I_i \right]$, i.e. a linear system $(I - A)p_b = c$. Its null space is the constants and it is not exactly compatible (curl $\mathbf{G} \neq 0$), so it is solved with the same zero-mean augmented system as above.
4. **Interior stage.** $p(P) = \text{mean}_k \left[p_b(E_k) + \int \mathbf{G} \cdot \mathbf{e}_k \, ds \right]$.

The original paper uses a virtual boundary with parallel rays and iterates the boundary values. This version keeps the principle (many directions, boundary values consistent with the rays) but starts the rays at each node.

Cost. $O(N \times \text{nodes} \times \text{ray length})$: about 0.4 s at 64^2 against 15 ms for Poisson. That is why the noisy grid study runs omni only up to 64^2.

Chapter 7

Iterative solvers, norms and convergence

The default Poisson solver is a sparse direct factorisation (`scipy.sparse.linalg.spsolve`). The course (Lecture 11) asks how iterative solvers behave, how to judge convergence, and how to measure residuals. The project implements all of this on the same matrix.

7.1 Norms of a residual

For a residual vector r with n entries:

Norm	Formula	Meaning
L_1	$\sum \lvert r_i \rvert$	total absolute imbalance
L_2	$\sqrt{\sum r_i^2}$	Euclidean length; large entries dominate
L_∞	$\max \lvert r_i \rvert$	worst cell only
L_{1n}	$\frac{1}{n}\sum \lvert r_i \rvert$	mean imbalance per cell
L_{2n}	$\sqrt{\frac{1}{n}\sum r_i^2}$	RMS imbalance per cell
scaled	$L_{2n}(r)/S$	relative imbalance ($S = L_{2n}(b)$)

For a residual of constant size c on n cells, $L_1 = nc$ and $L_2 = \sqrt{n}\,c$ grow with the grid while $L_{1n} = L_{2n} = c$ do not. A fixed tolerance on an unnormalised norm is therefore harder to meet the finer the grid; the project measures exponents 0.99 (L_1) and 0.50 (L_2) vs about 0 for the normalised norms. Always $L_{1n} \le L_{2n} \le L_\infty$.

Lecture example (a unit test): $r = (0.0001, 0.0004, 0.0006, 0.002)$ gives $L_1 = 0.0031$, $L_2 = 0.00213$, $L_{2n} = 0.00106$.

With $p^0 = 0$ the first residual is b, so the scaled residual starts at exactly 1, and "tol $= 10^{-8}$" means "reduce the imbalance by 10^8".

7.2 Jacobi is pseudo-time marching

Jacobi updates $p^{k+1} = p^k + \omega D^{-1}(b - Ap^k)$ with $D = \operatorname{diag} A$. For $\Delta x = \Delta y = h$ every diagonal entry is $-4/h^2$, so

$$p^{k+1} = p^k + \underbrace{\omega \frac{h^2}{4}}_{\Delta\tau} \left(\nabla_h^2 p^k - f \right), \tag{7.1}$$

which is explicit Euler time-stepping of the pseudo-transient equation $\partial p/\partial \tau = \nabla^2 p - f$ toward its steady state $\nabla^2 p = f$.

Stability. Each eigenmode is multiplied by $1 + \Delta\tau\lambda$, with $\lambda \in [-8/h^2, 0]$ for the 5-point Laplacian. Stability needs $|1 - 8\Delta\tau/h^2| \le 1$, i.e. $\Delta\tau \le h^2/4$, which is exactly classic Jacobi ($\omega = 1$).

Classic Jacobi stalls on the all-Neumann problem

At $\Delta\tau = h^2/4$ the checkerboard mode $(-1)^{i+j}$ (eigenvalue $-8/h^2$) is multiplied by -1 every step: it flips sign and never decays. With all-Neumann walls the checkerboard is an *exact* eigenvector, so the residual levels off at a floor (4.2×10^{-8} at 64^2, above tol 10^{-8}). The project uses $\omega = 0.95$ by default (amplification -0.9), which converges at a 5 % cost.

Convergence rate. The smoothest mode decays by $1 - \omega\mu_{min}$ per step with $\mu_{min} \approx (\pi h/L)^2/4$. Reducing the residual by a factor F takes about $\ln F/\mu_{min} \propto (L/h)^2 \propto N$ iterations: Jacobi's count grows linearly with the number of unknowns.

7.3 Gauss–Seidel and SOR

Gauss–Seidel uses the newest values as soon as they are available; SOR over-relaxes the update by ω. The code uses **red-black ordering** (all nodes with $i + j$ even, then all odd), which vectorises each half-sweep and is "consistently ordered", so Young's theory applies:

- $\rho_{GS} = \rho_J^2$: Gauss–Seidel needs half the Jacobi iterations (measured 12 328 vs 25 366 at 64^2).
- SOR is optimal at $\omega_{opt} = 2/\left(1 + \sqrt{1 - \rho_J^2}\right)$ with $\rho_{SOR} = \omega_{opt} - 1$; the iteration count then scales like $\sqrt{N}$ instead of N.

Neumann vs Dirichlet. The textbook formula $\omega_{opt} = 2/(1 + \sin(\pi h/L))$ assumes Dirichlet walls ($\rho_J = \cos \pi h/L$). With all-Neumann walls the slowest non-constant mode varies in one direction only, $\rho_J = \cos^2(\pi h/2L)$, so the optimum is larger: 1.932 instead of 1.905 at 64^2. The measured best ω is 1.9319 (314 iterations); the Dirichlet formula needs 628.

Kahan's theorem: $\rho_{SOR} \geq |\omega - 1|$, so SOR cannot converge for $\omega \geq 2$. The project shows $\omega = 2$ (no decay) and $\omega = 2.1$ (divergence to 10^{10} in 145 iterations).

7.4 Conjugate gradient

CG minimises the error in the energy norm over a growing Krylov space. It needs a symmetric positive (semi-)definite matrix, so it runs on $S = -WA$ (same equations, rows scaled by their trapezoidal weights) with a Jacobi preconditioner. Its iteration count scales like $\sqrt{\kappa(S)} \sim L/h \propto \sqrt{N}$. For the singular zero-mean problem it converges from $p^0 = 0$ because the right-hand side is made compatible.

7.5 Residual vs change: when is a solution converged?

For Jacobi, $p^{k+1} - p^k = \omega D^{-1} r^k$, so $\|p^{k+1} - p^k\| = \Delta\tau \|r^k\|$ with $\Delta\tau \approx 3 \times 10^{-4}$ at 64^2: the change is 10^3–10^4 times smaller than the residual. Worse, the true error is $\|p^k - p^*\| \approx \|p^{k+1} - p^k\|/(1 - \rho)$, and $1/(1 - \rho)$ is huge when convergence is slow.

Do not stop on a small change alone

A small change per iteration does *not* mean convergence when convergence is slow. In the project, the change first drops below 10^{-6} at iteration 6 697 while the scaled residual is still 6.1×10^{-4} and the true error 1.7×10^{-3}. The solvers therefore stop on the scaled residual. The same logic applies to declaring a CFD run "steady".

Viva questions

1. *Show that Jacobi is explicit pseudo-time marching.* $D^{-1} = -h^2/4$, so $p^{k+1} = p^k + \Delta\tau(\nabla_h^2 p^k - f)$ with $\Delta\tau = \omega h^2/4$.
2. *Why does classic Jacobi stall here but not with Dirichlet walls?* The Neumann checkerboard mode has amplification exactly -1; Dirichlet walls shift the extreme eigenvalue inside the stable range.
3. *Why is SOR so much faster?* Over-relaxation moves the spectral radius from $1 - O(h^2)$ to $1 - O(h)$: iterations $\propto \sqrt{N}$ instead of N.
4. *Why not use CG on A directly?* A (ghost-point form) is not symmetric; $-WA$ is.
5. *Why normalise residual norms?* So that the same physical imbalance gives the same number on any grid.

Part III

The virtual experiment

Chapter 8

Test flows, synthetic PIV and error metrics

To measure the error of a reconstruction we need flows whose pressure is known exactly. The project uses two exact Navier–Stokes solutions (plus a convected variant) and a CFD simulation, then degrades their velocity like a real PIV system.

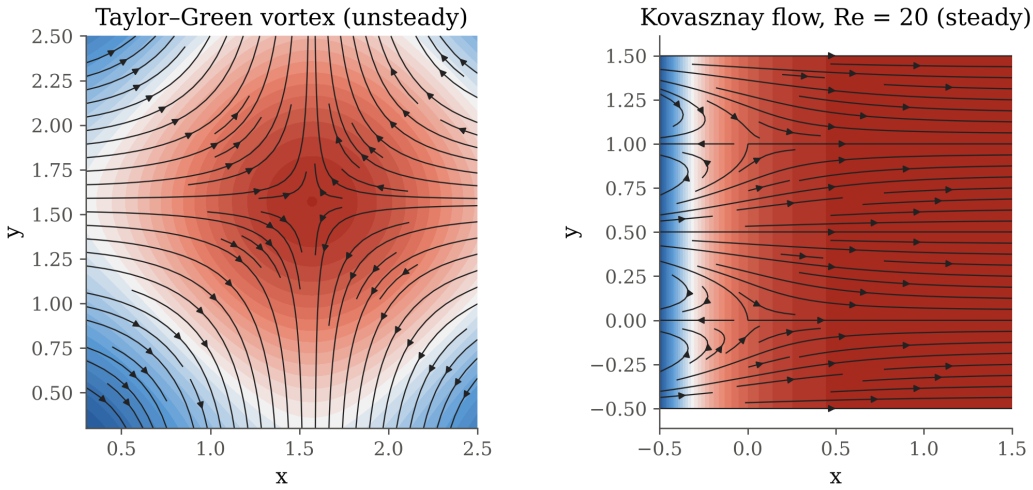


Figure 8.1: The two exact test flows: pressure (colour) and streamlines. Left: Taylor–Green on the window $[0.3, 2.5]^2$. Right: Kovasznay flow at Re = 20 on $[-0.5, 1.5]^2$.

8.1 Taylor–Green vortex

$$u = \cos x \sin y\, F(t), \quad v = -\sin x \cos y\, F(t), \quad p = -\tfrac{\rho}{4}(\cos 2x + \cos 2y)F(t)^2, \quad F = e^{-2\nu t}. \tag{8.1}$$

Why it is exact. Each velocity component is an eigenfunction of the Laplacian, $\nabla^2 u = -2u$, so $\partial \mathbf{u}/\partial t = -2\nu\mathbf{u} = \nu\nabla^2\mathbf{u}$: the unsteady and viscous terms cancel exactly. The convective term is then a pure gradient, balanced by $-\nabla p/\rho$:

$$(\mathbf{u}\cdot\nabla)\mathbf{u} = \tfrac{1}{2}\big(\sin 2x,\ \sin 2y\big)F^2 = -\nabla p/\rho.$$

So $\mathbf{G} = \nabla p$ comes entirely from convection: a good test of the convective term.

Settings. $\nu = 0.01$, snapshots at $t = 1 \pm \Delta t$ with $\Delta t = 0.05$.

- **Main window** $[0.3, 2.5]^2$: deliberately non-periodic, so the Neumann boundary treatment is actually tested (on a full period the boundary terms would cancel).
- **Core window** $[\pi - 1.1, \pi + 1.1]^2$: the main window contains no vortex core (its centre is a saddle, a pressure maximum), so vortex-core metrics use this second window centred on the core at (π, π).

Convected Taylor–Green. Navier–Stokes is Galilean invariant: if $(\mathbf{u}, p)$ is a solution, so is $\mathbf{u}'(\mathbf{x}, t) = \mathbf{c} + \mathbf{u}(\mathbf{x} - \mathbf{c}t, t)$, $p' = p(\mathbf{x} - \mathbf{c}t, t)$. With amplitude $A = 0.4$ and $\mathbf{c} = (1, 0)$ the vortex pattern is swept through the window, so $\partial\mathbf{u}/\partial t = O(1)$ as in a real wake. This is the case where the Eulerian/Lagrangian comparison is meaningful (vortex speed $w = 0.4 <$ convection speed $c = 1$).

8.2 Kovasznay flow

Kovasznay (1948) found an exact steady solution resembling the flow behind a grid:

$$u = 1 - e^{\lambda x}\cos 2\pi y, \quad v = \frac{\lambda}{2\pi}e^{\lambda x}\sin 2\pi y, \quad p = \frac{\rho}{2}\left(1 - e^{2\lambda x}\right), \quad \lambda = \frac{Re}{2} - \sqrt{\frac{Re^2}{4} + 4\pi^2}. \tag{8.2}$$

- Continuity: $u_x = -\lambda e^{\lambda x}\cos 2\pi y$ and $v_y = \lambda e^{\lambda x}\cos 2\pi y$ cancel. ✓
- Substituting into the steady equations gives $\lambda^2 - Re\,\lambda - 4\pi^2 = 0$; the negative root gives a wake that decays downstream: $\lambda \approx -1.81$ at Re = 20 and -0.96 at Re = 40.
- The flow is steady, so $D\mathbf{u}/Dt = (\mathbf{u}\cdot\nabla)\mathbf{u}$, and $\mu = \rho/Re$ is large: the viscous term matters. Kovasznay therefore tests the viscous term, which Taylor–Green does not.
- Its pressure depends on x only, with a steep gradient near the inflow edge $x = -0.5$.

8.3 The synthetic PIV model

`piv/degrade.py` turns an exact field into PIV-like data in three steps, applied in this order:

1. **Interrogation-window averaging.** The truth is sampled 4× finer than the PIV grid (with a margin), and a box filter of width W is applied to u and v of every snapshot. For even widths the plain moving average would be off-centre by half a cell; the code averages the two half-shifted windows, which gives a centred trapezoidal box of width exactly W.
2. **Subsampling** to the PIV spacing h (every 4th fine node), then cropping the margin, so the PIV nodes are exactly `linspace(x0, x1, n)`.
3. **Noise.** Independent Gaussian noise $N(0, (\sigma U)^2)$ on u and v at every node and snapshot, with a fixed seed.

The *reference pressure is never filtered*: it is only subsampled, so the measured error includes the bias that window averaging causes.

Baseline. 64×64 vectors, $W = h$, $\sigma = 1\,\%$ of U, $\Delta t = 0.05$. Each study varies one parameter around this baseline.

Fixed noise level across window sizes

In real PIV a larger window contains more particles, so its random error is *smaller*. The model keeps σ fixed (as the project specification asked), so here W only adds bias. Making σ decrease with W would reproduce the real resolution–noise trade-off; this is listed as future work.

8.4 Denoising

Two optional filters act on u and v before differentiation:

- **Gaussian filter** (standard deviation 1 grid point): transfer function $e^{-(ksh)^2/2}$. It smooths everything, including real curvature.
- **2-D Savitzky–Golay** (5×5 window, quadratic): at each node, a least-squares fit of a polynomial in x, y, evaluated at the node (via the pseudo-inverse of the Vandermonde matrix). It reproduces quadratics exactly, also at the edges where the window is shifted inwards, so it biases smooth peaks less than the Gaussian.

8.5 Error metrics

Pressure is known only up to a constant, so both fields are demeaned first ($p' = p - \bar{p}$):

$$\varepsilon_{L2} = \frac{\|p' - p'_{ref}\|_2}{\|p'_{ref}\|_2}, \qquad \varepsilon_{max} = \frac{\max|p' - p'_{ref}|}{\frac{1}{2}\rho U^2}. \tag{8.3}$$

Also reported: the error map $|p' - p'_{ref}|$; a line profile through the vortex core; and the **core ratio**, reconstructed core pressure (minimum within $3h$ of the true core) divided by the true one. A ratio below 1 means the suction is under-predicted.

8.6 Experiment design

- **One factor at a time** around the baseline, so each effect is attributed cleanly (a local sensitivity study).
- **Ten noise seeds** per point; results are mean $\pm$ standard deviation. The standard error of the mean falls as $1/\sqrt{N}$.
- **Common random numbers:** the same seeds are reused across parameter values, so differences between parameter values are less noisy than the scatter itself.
- Runs are deterministic: re-running reproduces every number.

Chapter 9

The code

9.1 Architecture

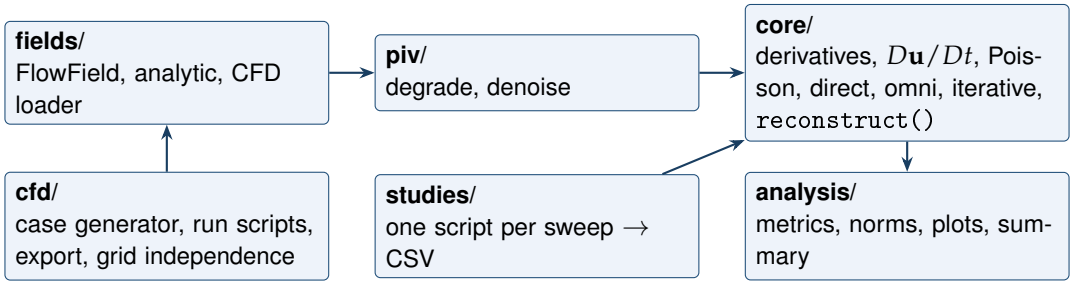


Figure 9.1: Package structure. Every data source returns a `FlowField`; every algorithm consumes one.

- `FlowField` (`fields/field.py`) is the common interface: x, y, t and arrays u, v of shape $[n_t, n_y, n_x]$, plus ρ, μ, the reference pressure and U_{ref}. It checks that the grids are uniform, because every stencil assumes constant spacing.
- `reconstruct()` (`core/pipeline.py`) is the single entry point: it computes $\mathbf{G}$, then calls the chosen method, and returns the pressure plus diagnostics (compatibility residual, timings, solver history).
- `studies/` contains one script per parameter sweep; each writes a CSV row per reconstruction and saves its figures.
- `analysis/summary.py` writes `results/SUMMARY.md` from the CSVs, so no number in the report is typed by hand.

9.2 Pseudo-code

```
for case in [TaylorGreen, ConvectedTG, Kovasznay20, Kovasznay40, CFD]:
    truth = exact_or_cfd(case)                  # u, v, p on a 4x finer grid
    for param in sweep(h | sigma | dt | BC | method | W | filter):
        for seed in range(10):
            F  = degrade(truth, W, h, sigma, seed)  # box filter, subsample, noise
            F  = denoise(F)                       # optional
            a  = DuDt(F, eulerian | lagrangian)
            G  = -rho*a + mu*lap(u)
            p  = poisson(G) | direct(G) | omni(G)
            eps  = L2(p' - p'_ref) / L2(p'_ref)   # p' = p - mean(p)
        save mean, std of eps -> CSV -> figures -> SUMMARY.md
```

9.3 Inside the Poisson solve

```
f = div(G)                                  # or -rho(u_x^2 + 2 u_y v_x + v_y^2)
b = f + ghost-point terms +/-2G/h on the edges (both at corners)
R = trapz(f) - contour(n.G)                 # compatibility residual, reported
if Dirichlet edges:   rows -> p_k = value
elif pin == "node":   row k -> p_k = 0
else:                 solve [[A, 1], [1^T, 0]] [p, lam] = [b, 0]   # zero mean
p = spsolve(...)   or   Jacobi / GS / SOR / CG on the same system
```

9.4 Tests: how we know the code is right

87 unit tests (`python -m pytest -q`, about 15 s) check every component against an exact answer:

- **Derivatives:** second-order slopes in the max norm, including boundary nodes; exact on quadratics and cubics.
- **Poisson:** the manufactured solution $p = \sin x \cos y$ converges at order 2 for pin-mean, pin-node, direct and CG; the null space, $w^T A = 0$, symmetry of WA, and the discrete compatibility identity.
- **Taylor–Green and Kovasznay:** $\mathbf{G} = +\nabla p$ to second order; full reconstruction second order for all three methods; without $\mu\nabla^2\mathbf{u}$ Kovasznay does not converge.
- **Methods:** all exact for linear p; agree on clean data; noisy data show path dependence and omni/Poisson beat a single path.
- **PIV model and I/O:** box filter centred and normalised; noise seeding; Savitzky–Golay preserves quadratics; OpenFOAM export parsers; Strouhal number from a synthetic force file; Richardson extrapolation recovers order 2.
- **Convergence add-on:** norms (the lecture example), Jacobi = pseudo-time, classic-Jacobi stall, SOR divergence, all iterative solvers match `spsolve`, slope ≈ 1 for `order=1` and ≈ 2 for `order=2`.

Viva questions

1. *Why generate the truth finer than the PIV grid?* A PIV vector is an average over the window, so the underlying field must be resolved below h.
2. *Why fixed seeds?* Reproducibility, and common random numbers make differences between parameter values less noisy.
3. *How do you test second order?* Halve h repeatedly and fit the slope of $\log \varepsilon$ against $\log h$; the tests accept 1.8–2.3 because coarse grids are pre-asymptotic.
4. *Why a common `FlowField`?* Degradation, integration and metrics need not know whether data are analytic, CFD or turbulence-database data.

Part IV

Results and what they mean

Chapter 10

Reconstruction results

All numbers are means over 10 noise seeds from `results/*.csv`. The baseline is $n = 64$, $W = h$, $\sigma = 1\,\%$, $\Delta t = 0.05$, Poisson with all-Neumann edges and the zero-mean constraint, Eulerian $D\mathbf{u}/Dt$.

10.1 Verification: grid convergence on clean data

First the code must be shown correct, with no noise and no window averaging (exact point samples).

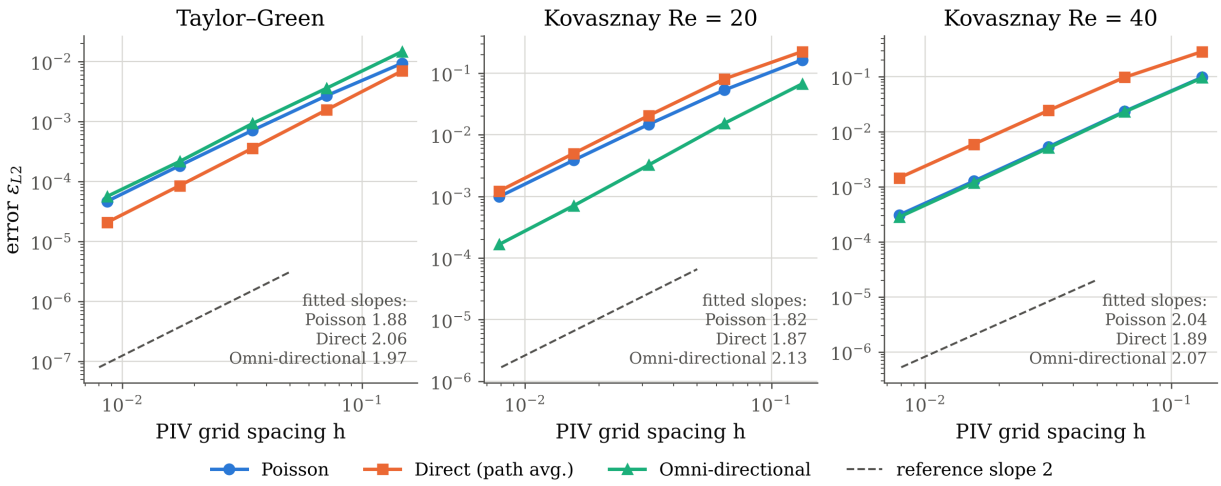


Figure 10.1: Error vs PIV spacing on clean data for the three test flows and three methods; dashed line: slope 2.

Fitted slope of ε_{L2} vs h (16^2–256^2)	Poisson	Direct	Omni
Taylor–Green	1.88	2.06	1.97
Kovasznay Re = 20	1.82	1.87	2.13
Kovasznay Re = 40	2.04	1.89	2.07

What it shows. All three methods are second-order accurate on all three flows, as the $O(h^2)$ stencils and trapezoidal rule predict. Between the two finest grids the local Poisson slopes are 1.96–2.03; fits over all grids are slightly lower because the coarsest grids are pre-asymptotic. The finest-grid errors are tiny (4.7×10^{-5} for Poisson on Taylor–Green at 256^2). Kovasznay errors are larger at a given h because its pressure varies steeply near the inflow edge ($e^{2\lambda x}$). The code is verified before any noise is added.

10.2 Noise vs resolution: an optimum spacing

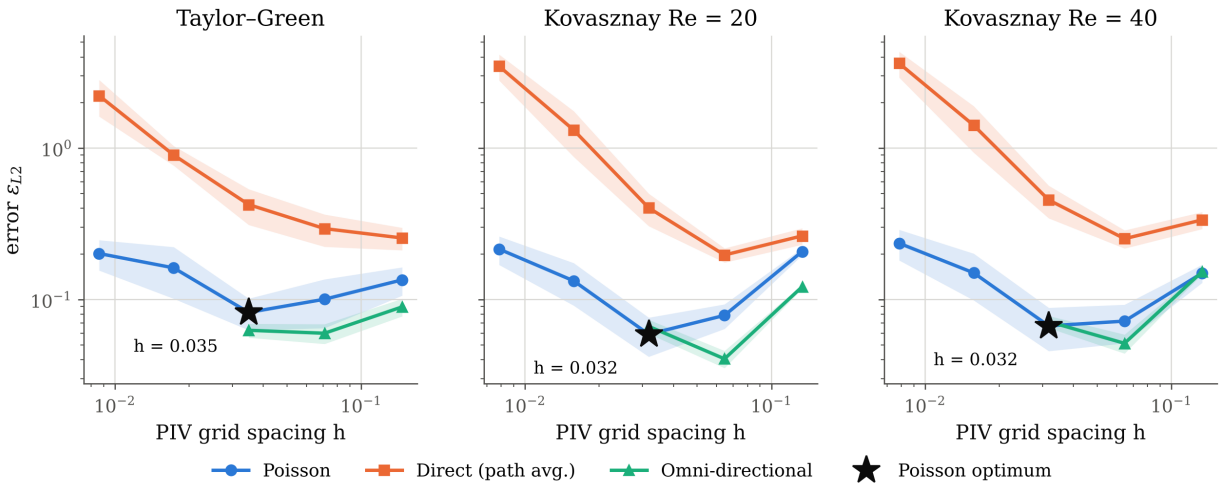


Figure 10.2: Error vs spacing with $\sigma = 1\,\%$, $W = h$ (mean $\pm$ std of 10 seeds). Stars: the Poisson optimum.

n	h (TG)	Taylor–Green	Kovasznay 20	Kovasznay 40
16	0.147	0.135	0.208	0.150
32	0.071	0.101	0.079	0.072
64	**0.035**	**0.082**	**0.059**	**0.067**
128	0.017	0.162	0.133	0.150
256	0.009	0.202	0.216	0.236

Table 10.1: Poisson ε_{L2} with 1 % noise. The minimum is at $n = 64$ for all three flows.

What it shows and why. With noise, refining the grid stops helping: the Poisson error is lowest at $h \approx 0.032$–0.035 ($n = 64$) for all three flows and rises again on finer grids. This is the trade-off of Eq. (4.4). Truncation error falls as h^2, but noise is amplified as σ/h by the convective term and as $\mu\sigma/h^2$ by the viscous term. At $h = 0.035$ the viscous noise is already comparable to the convective noise, and it grows fastest.

Direct integration only gets worse as the grid is refined (Taylor–Green 0.26 $\rightarrow$ 2.2): a path crosses $\sim 1/h$ noisy steps and its error grows like a random walk. On Kovasznay its optimum is at the coarser $h \approx 0.065$.

Practical consequence

For noisy PIV data there is a best vector spacing. Asking for the finest possible PIV grid can make the pressure *worse*. The optimum moves to coarser grids as the noise level rises.

10.3 Noise sensitivity and choice of method

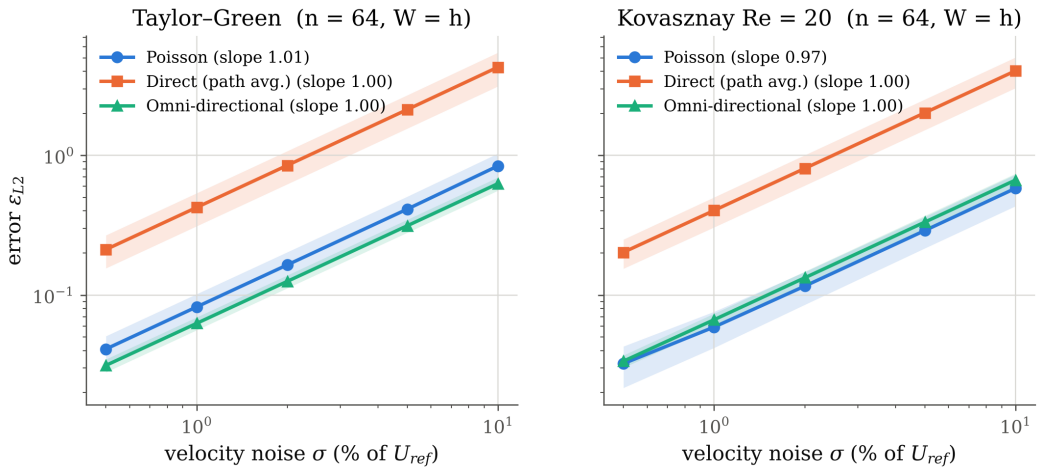


Figure 10.3: Error vs noise level ($n = 64$, $W = h$); the slopes in the legend are log–log fits.

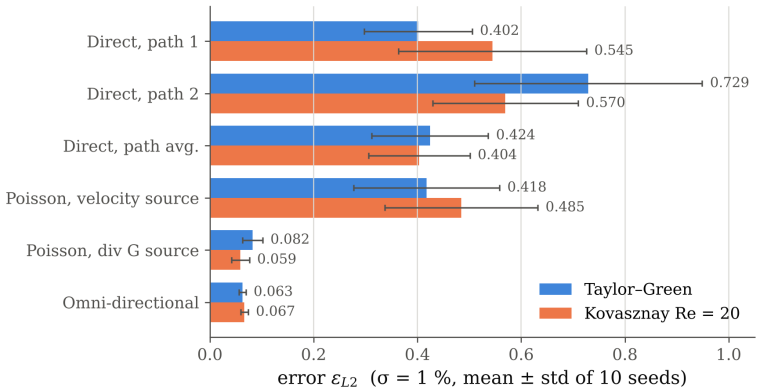


Figure 10.4: All integration variants at $\sigma = 1\,\%$ (mean $\pm$ std of 10 seeds).

Method	ε_{L2} at $\sigma = 1\,\%$ TG	Kov 20	log–log slope vs σ TG	Kov 20	Time per reconstruction
Direct (path avg.)	0.424	0.404	1.00	1.00	0.3 ms
Poisson ($\nabla \cdot \mathbf{G}$ source)	0.082	0.059	1.01	0.97	15 ms
Omni-directional	0.063	0.067	1.00	1.00	0.41 s
Poisson (velocity source)	0.418	0.485	–	–	15 ms

1. **Error grows linearly with noise** (log–log slopes 0.97–1.01). The whole reconstruction is a linear operation on the velocity error (apart from the small quadratic convective term), so doubling the noise doubles the error.
2. **Direct integration is 5–7× worse.** A single path carries every error along it; Poisson (elliptic: every point sees the whole domain) and omni (average of 32 directions) average errors out.
3. **Omni and Poisson are equally accurate** (each wins one flow) but omni is about 27× slower. Poisson is the practical choice.
4. **Path 1 and path 2 differ** (Taylor–Green 0.40 vs 0.73). This is not a bug: mirroring the data ($x \leftrightarrow y$) swaps the two errors exactly. Noise in the convective and viscous terms comes

from the same snapshot, so the terms are correlated; whether they add or cancel along a path depends on the flow direction relative to the path.

5. **The velocity-source Poisson form is 5–8× worse with noise.** Its interior source multiplies noisy derivatives, while its boundary data still come from **G**, so source and boundary data no longer come from one vector field. The least-squares structure of Section 3.3.2 is lost. (With noise-free boundary data it was actually better: its weakness is the mismatch, not the identity.)

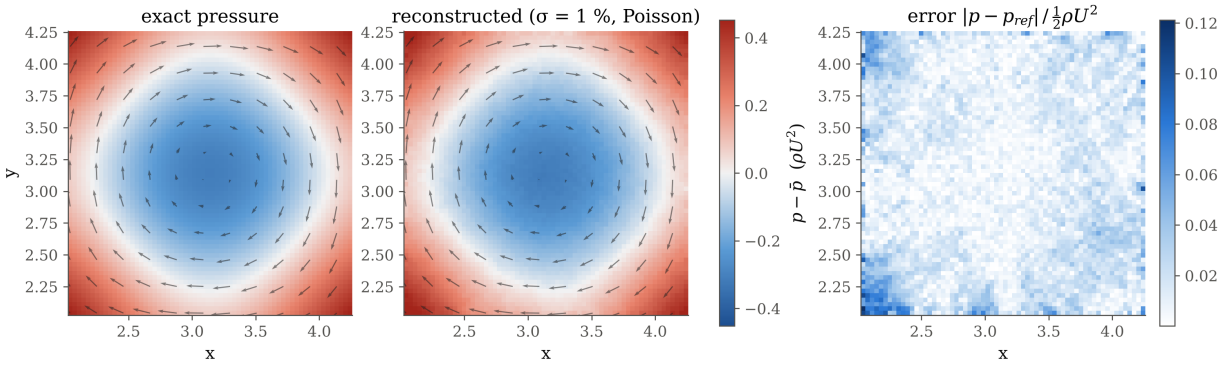


Figure 10.5: Taylor–Green core window: exact pressure, reconstruction from noisy PIV-like data ($\sigma = 1\,\%$, Poisson, $\varepsilon_{L2} = 0.061$ for this seed), and the error map. The vortex is recovered; the error is mostly random scatter, largest at the edges.

10.4 Boundary conditions for the Poisson solver

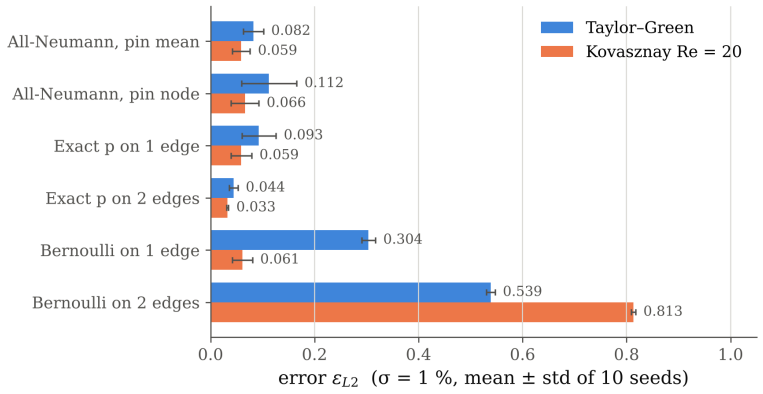


Figure 10.6: Poisson error for six boundary treatments at $\sigma = 1\,\%$.

Boundary treatment	TG noisy ε_{L2}	Kov 20 noisy ε_{L2}
All-Neumann, zero mean (baseline)	0.082	0.059
All-Neumann, pin one node	0.112	0.066
Exact p on 1 edge	0.093	0.059
Exact p on 2 opposite edges	**0.044**	**0.033**
Bernoulli on 1 edge	0.304	0.061
Bernoulli on 2 edges	0.539	0.813

1. **Known pressure on two opposite edges halves the error.** Neumann data $\mathbf{n} \cdot \mathbf{G}$ are themselves noisy derivatives, so their errors propagate inward; a Dirichlet edge anchors the

solution. Two opposite edges bracket the domain. (The "exact" option is an oracle standing in for wall pressure taps.)

2. **One edge helps little or even hurts**: the Neumann-data error is anchored on one side only and accumulates across the domain.
3. **Pinning a node is worse than the zero-mean constraint**: it dumps the incompatibility at one point (Section 5.3).
4. **Bernoulli edges fail** (up to 0.81): $p + \frac{1}{2}\rho|\mathbf{u}|^2 = \text{const}$ holds only in irrotational flow, and both flows are rotational everywhere. Use Bernoulli only at genuine free-stream edges.

10.5 Time step: Eulerian vs Lagrangian

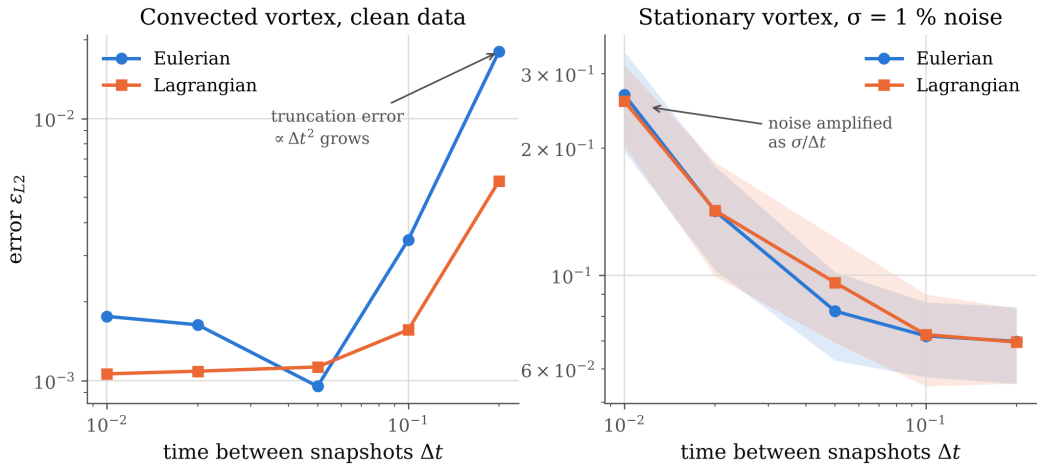


Figure 10.7: Error vs time between snapshots. Left: convected vortex, clean data. Right: stationary vortex with 1 % noise.

	Convected TG, clean		Stationary TG, $\sigma = 1\,\%$	
Δt	Eulerian	Lagrangian	Eulerian	Lagrangian
0.01	1.8×10^{-3}	1.1×10^{-3}	0.267	0.258
0.05	9.5×10^{-4}	1.1×10^{-3}	0.082	0.096
0.10	3.5×10^{-3}	1.6×10^{-3}	0.072	0.072
0.20	1.8×10^{-2}	5.8×10^{-3}	0.070	0.070

1. **Convected vortex (clean):** the Eulerian error grows $10\times$ from $\Delta t = 0.01$ to 0.2, because at a fixed point the pattern sweeps past and $\partial\mathbf{u}/\partial t$ changes quickly ($\propto c^3$). The Lagrangian scheme follows the particle; its error grows only with the vortex's own velocity ($\propto w^3$) and is $3\times$ lower at $\Delta t = 0.2$. This is exactly the w/c argument of Section 3.2.
2. **Stationary vortex (clean):** the Eulerian error does not depend on Δt (the flow barely changes at a point), while the Lagrangian error grows (to 4.2×10^{-3} at $\Delta t = 0.2$) because it traces curved particle paths with a straight step.
3. **Noisy data:** both schemes *improve* with larger Δt (0.27 $\rightarrow$ 0.07), because the noise in $\partial\mathbf{u}/\partial t$ scales as $\sigma/\Delta t$ and dominates over this range. The best Δt balances noise against truncation error, just like the best h.

10.6 Interrogation window and denoising

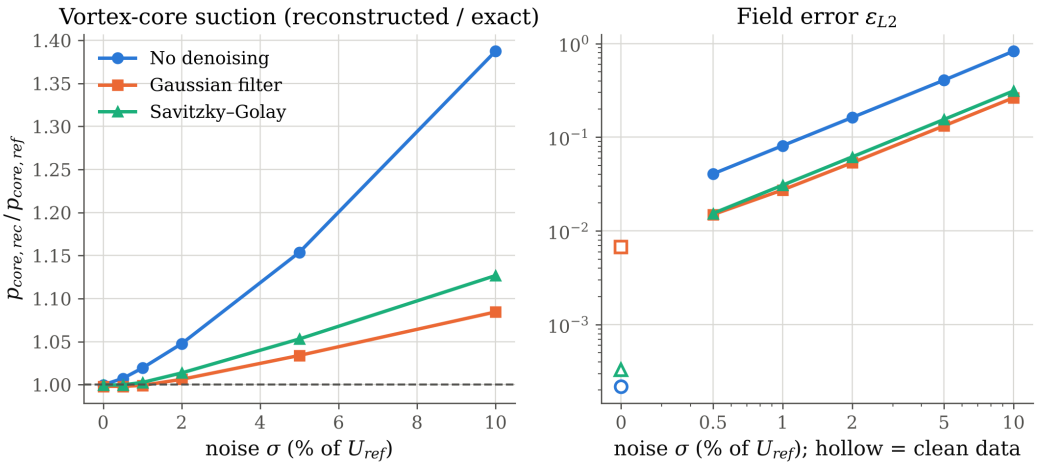


Figure 10.8: Taylor–Green core window. Left: reconstructed/exact vortex-core suction vs noise. Right: field error vs noise; hollow markers = clean data.

Window size W. Averaging over the window smooths velocity. On the Taylor–Green core the suction drops only 0.3 % at $W = 4h$, because the vortex (size $\sim \pi$) is much larger than the window ($4h \approx 0.14$). On Kovasznay, with its sharp gradients near the inflow, ε_{L2} rises from 0.015 to 0.061 (clean). The effect grows with W relative to the flow structure size, so it is expected to be much stronger for the cylinder-wake vortices.

σ (%)	None	Gaussian	Savitzky–Golay
0	0.0002	0.0068	0.0003
0.5	0.041	0.015	0.015
1	0.081	0.027	0.031
2	0.162	0.053	0.062
5	0.406	0.132	0.155
10	0.828	0.264	0.312

Table 10.2: Taylor–Green core window, ε_{L2} with and without denoising.

1. **Denoising cuts the noisy error about 3×** at every noise level from 0.5 %.
2. **On clean data it hurts**: the Gaussian filter raises the error from 0.0002 to 0.007 because it also smooths the true signal. Savitzky–Golay barely changes clean data (it preserves quadratics), but on Kovasznay both filters help only above 0.5–1 %.
3. **Noise biases the core, not just scatters it.** Without denoising the core suction is *over*-predicted, 1.39× at $\sigma = 10$ %. The convective term multiplies noisy quantities, and products of zero-mean noise do not average to zero; also the minimum of a noisy field is biased low (a selection bias). Filtering reduces both.

Smoothing trades noise for bias

Window averaging, denoising and larger Δt all reduce random error and add systematic error. The best setting depends on the noise level: none at $\sigma \approx 0$, moderate above about 0.5 %.

Chapter 11

Solver and convergence results

11.1 Residual histories

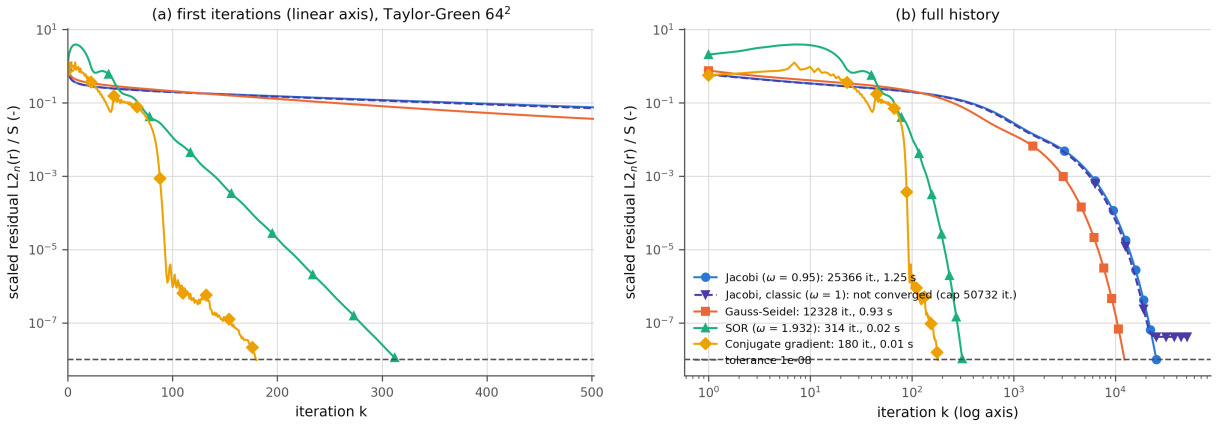


Figure 11.1: Scaled residual history of five iterative solvers on the Taylor–Green 64^2 Poisson system.

Solver (64^2, tol 10^{-8})	Iterations	Time
Jacobi ($\omega = 0.95$)	25 366	1.3 s
Gauss–Seidel (red-black)	12 328	0.93 s
SOR ($\omega = 1.932$)	314	0.024 s
Conjugate gradient	180	0.013 s
Jacobi, classic ($\omega = 1$)	stalls at 4.2×10^{-8}	–
Sparse direct (`spsolve`)	–	≈0.015 s

Gauss–Seidel needs half the Jacobi iterations ($\rho_{GS} = \rho_J^2$); SOR and CG need 80–140× fewer. Classic Jacobi levels off above the tolerance: the checkerboard stall of Section 7.2. All solvers give the same pressure as the direct solve (agreement $< 10^{-6}$).

11.2 Iterations vs grid size and SOR stability

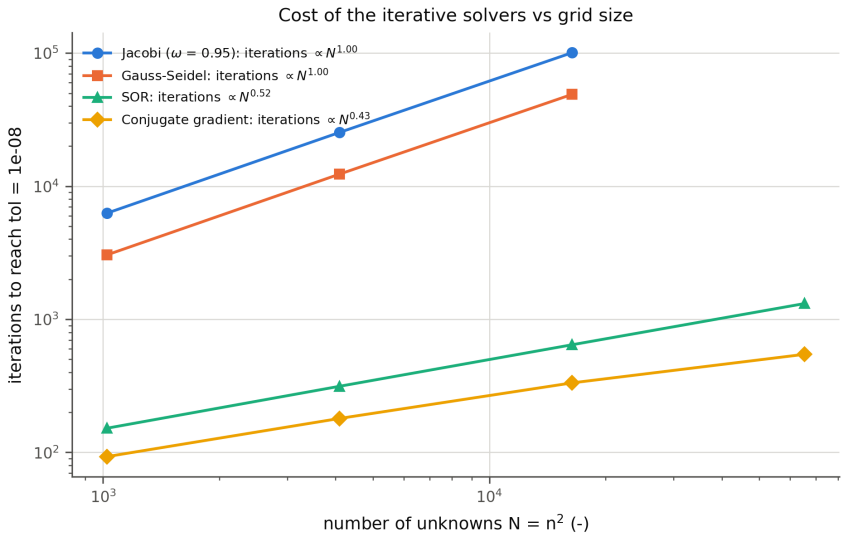


Figure 11.2: Iterations to converge vs number of unknowns N.

Fitted exponents: Jacobi and Gauss–Seidel $N^{1.00}$, SOR $N^{0.52}$, CG $N^{0.43}$, as the theory predicts (N for Jacobi/GS, $\sqrt{N}$ for optimal SOR and CG).

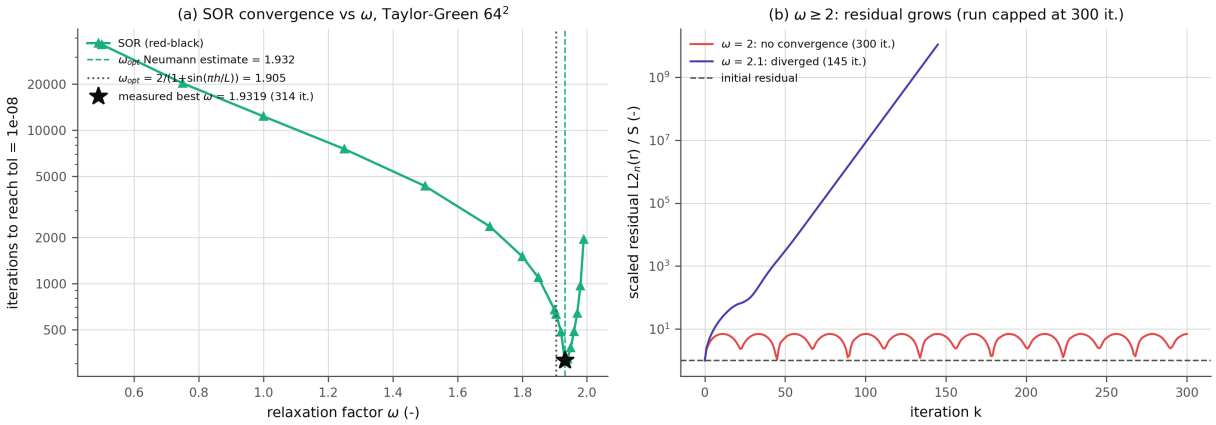


Figure 11.3: (a) SOR iterations vs ω; the measured optimum matches the Neumann estimate 1.932, not the Dirichlet formula 1.905. (b) $\omega = 2$ does not converge and $\omega = 2.1$ diverges (Kahan).

11.3 Residual vs change

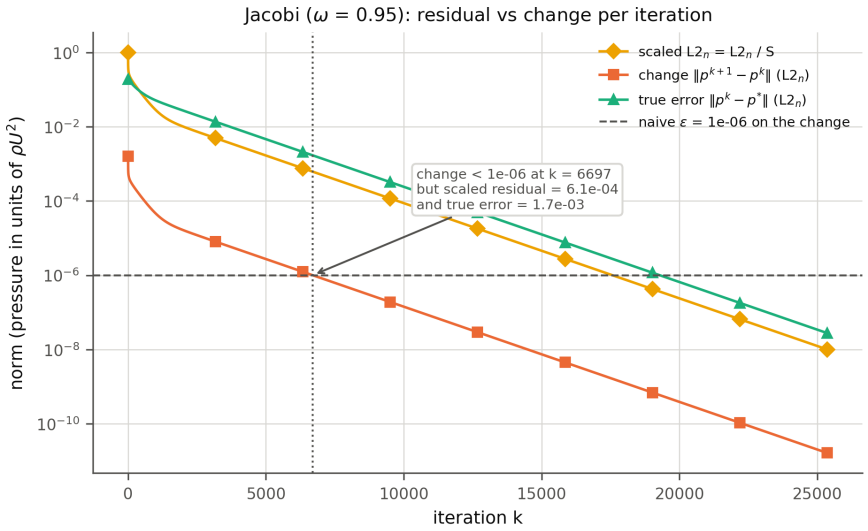


Figure 11.4: Jacobi: scaled residual, change per iteration and true error. When the change first drops below 10^{-6} the solution is still far from converged.

11.4 Norms across grids

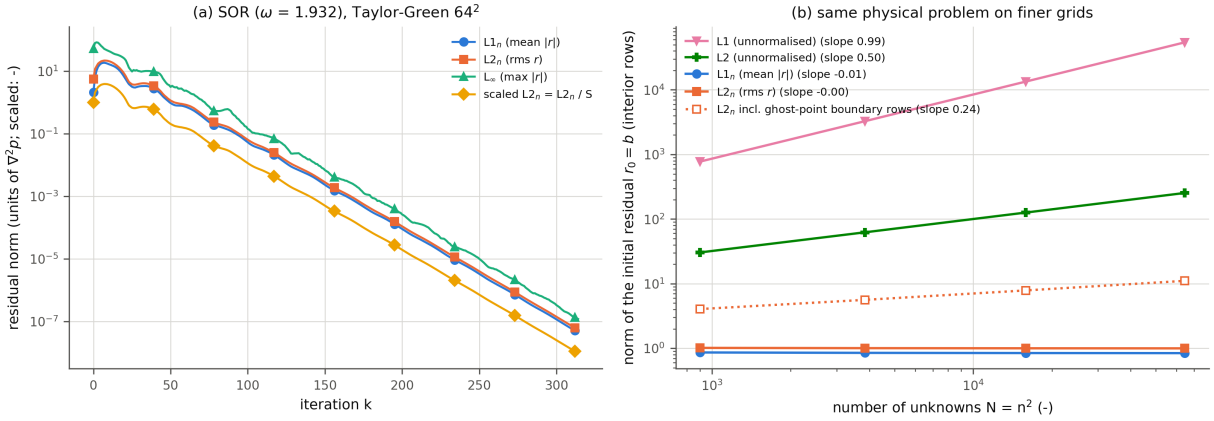


Figure 11.5: Different norms of the same residual. Unnormalised L_1, L_2 grow with grid size ($N^{0.99}$, $N^{0.50}$); normalised norms stay flat.

11.5 First- vs second-order schemes

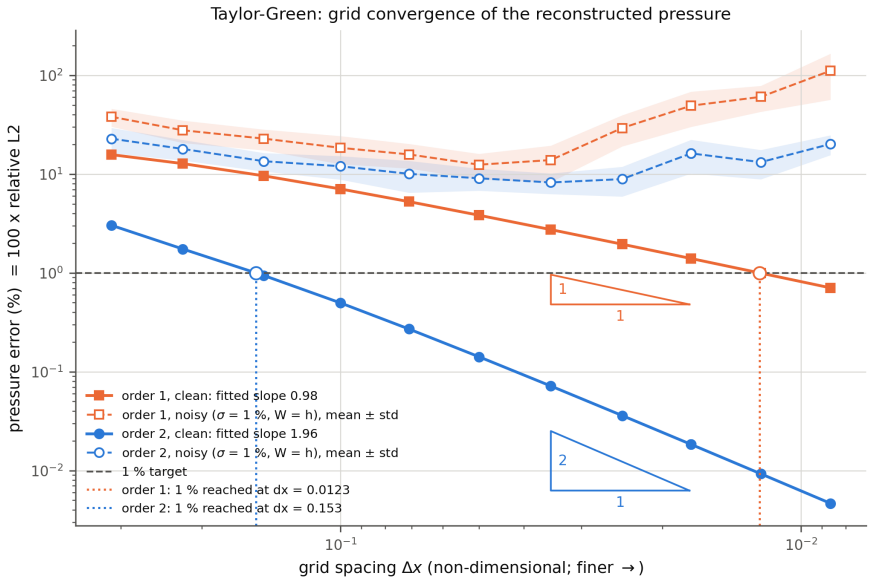


Figure 11.6: Grid convergence of the reconstructed pressure on Taylor–Green with first- and second-order stencils; dashed: 1 % noise.

Case	Slope (finest 5 grids) order 1	order 2	Δx for 1 % error order 1	order 2
Taylor–Green	0.98	1.96	0.0123	0.153
Kovasznay Re = 20	1.00	1.96	0.0033 (extrap.)	0.026

The second-order scheme reaches 1 % error on a grid about 12× coarser in each direction (Taylor–Green). To halve the error, order 1 must refine Δx by 2, order 2 only by $\sqrt{2}$. With 1 % noise both curves level off and rise again; the second-order scheme stays lower.

Why order 1 gives slope 1 even with an $O(1)$ boundary error. The first-order Neumann row has an $O(1)$ local truncation error, but it acts only on a layer of width $O(h)$, so the global error is $O(h)$, the same argument that makes the second-order scheme's $O(h)$ boundary rows globally $O(h^2)$.

Part V

Simulation, literature and conclusions

Chapter 12

The CFD case: a cylinder wake at Re = 100

Exact solutions are smooth and simple. A real application is an unsteady wake with vortices shed from a body. The project therefore simulates the 2-D laminar flow past a circular cylinder at $Re = U_\infty D/\nu = 100$ in OpenFOAM, samples it in a PIV-like window, and runs the same reconstruction on that data with the CFD pressure as the reference.

The three meshes were run with the OpenFOAM v2606 MinGW build on native Windows (4 cores, MS-MPI), and the solution was inspected in ParaView 6.2 (Figure 12.2).

12.1 Why this flow

- Above $Re \approx 47$ the steady wake becomes unstable and vortices are shed alternately from the two sides: the **Kármán vortex street**. The flow is periodic with Strouhal number $St = fD/U \approx 0.165$ at $Re = 100$.
- Below $Re \approx 190$ the wake stays two-dimensional (Williamson 1996), so a 2-D simulation is physically valid and planar PIV would miss nothing.
- The shed vortices are *convected* at nearly the free-stream speed while they spin more slowly: exactly the $w < c$ situation where the Lagrangian material derivative should win.

12.2 Domain, mesh and boundary conditions

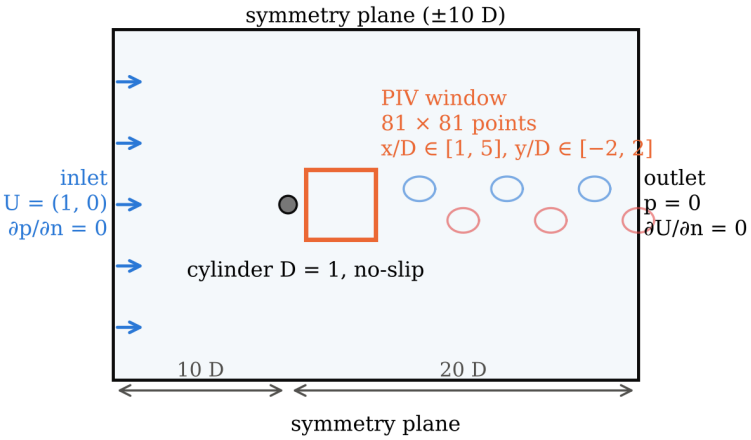


Figure 12.1: Computational domain (not to scale), boundary conditions and the PIV sampling window.

- **Non-dimensional set-up:** $D = 1$, $U_\infty = 1$, $\nu = 0.01$, $\rho = 1$.
- **Domain:** $10\,D$ upstream, $20\,D$ downstream, $\pm 10\,D$ laterally; one cell thick in z with `empty` front and back patches (OpenFOAM's way of making a case 2-D).
- **Boundary conditions:**
  - inlet: $\mathbf{u} = (1, 0)$, $\partial p/\partial n = 0$;

  - outlet: $\partial \mathbf{u}/\partial n = 0$, $p = 0$;
  - sides: `symmetryPlane` (zero normal velocity and zero shear);
  - cylinder: no-slip.

- **Blockage:** the side walls are $20\,D$ apart, so the blockage is $D/H = 5\,\%$. It slightly accelerates the flow past the cylinder and raises C_D and St above unbounded-domain values.
- **Mesh:** an O-grid around the cylinder (body-fitted, near-orthogonal cells clustered radially at the wall for the boundary layer) out to the square $[-2, 2]^2$, plus 8 rectangular blocks. Generated with `blockMesh` from a dictionary written by `cfd/generate_cases.py`, which also checks the block topology.

Mesh	Cells	Cylinder tangential cell	First radial cell	Wake Δy
Coarse	6 690	0.033	0.022	0.167
Medium	26 760	0.016	0.011	0.083
Fine	107 040	0.008	0.0055	0.042

Table 12.1: The three meshes: each level halves the cell size in both directions ($4\times$ the cells).

12.3 Solver and numerics

- **Solver:** `pimpleFoam`, laminar, run in PISO mode (`nOuterCorrectors 1`, `nCorrectors 2`, one non-orthogonal corrector).
- **Time scheme:** `backward` (second-order implicit). **Convection:** `linearUpwind` (second-order upwind-biased).
- **Time step:** adaptive, keeping the maximum Courant number $Co = |u|\Delta t/\Delta x \leq 0.5$ (with $\Delta t \leq 0.01$); the run goes to $t = 200$, about 33 shedding periods.
- **Start-up kick:** a perfectly symmetric mesh and inflow can delay shedding for a long time. The inlet carries $v = 0.1$ for $t < 5$, ramped to 0 at $t = 6$; from $t = 6$ on it is exactly $\mathbf{u} = (1, 0)$.

How PISO works, and why it connects to this project. In incompressible flow there is no equation *for* pressure; pressure is whatever makes the velocity divergence-free. Each time step, PISO:

1. solves the momentum equation with the old pressure (a predicted velocity that is not divergence-free);
2. takes the divergence of the momentum equation to get a **pressure Poisson equation**, $\nabla \cdot (\frac{1}{a}\nabla p) = \nabla \cdot \mathbf{H}$, and solves it;
3. corrects the velocity and fluxes with the new pressure gradient, and repeats steps 2–3 (here two pressure correctors per step, `nCorrectors 2`; the pressure equation is solved with the GAMG multigrid solver).

So the CFD solver itself recovers pressure from velocity through a Poisson equation every time step, the same mathematical step the reconstruction performs on measured data. The difference: the CFD solver has exact boundary conditions and noise-free velocity; PIV has neither.

12.4 Outputs and how they are processed

- **Forces:** the `forceCoeffs` function object writes $C_D(t)$ and $C_L(t)$ every step.

- **PIV window:** on the fine mesh, a run-time `sets` function object samples $\mathbf{u}$ and p on an 81×81 grid, $x/D \in [1,5]$, $y/D \in [-2,2]$, every 0.05 time units for $t \in [175,200]$ (about 4 shedding periods). Sampling at run time avoids writing about 500 full-field dumps.
- `cfd/export_window.py` converts the samples into one array file, which the studies load as a `FlowField`.

12.4.1 Strouhal number, drag and lift

$C_L(t)$ is resampled on a uniform time grid (the time step is adaptive), multiplied by a Hann window (to limit spectral leakage from a non-integer number of periods), zero-padded and Fourier transformed. The peak frequency, refined by parabolic interpolation, gives $St = fD/U_\infty$. Mean C_D and the C_L amplitude $(\max - \min)/2$ are taken over the last 80 time units.

12.4.2 Is the wake fully developed? Periodicity, not steady state

A periodic flow never satisfies a steady-state criterion $\|\mathbf{u}^{n+1} - \mathbf{u}^n\| < \varepsilon$, and its solver residuals level off instead of falling to zero (each step starts from a changed flow). The project instead cuts $C_L(t)$ into cycles at upward zero crossings and declares the flow periodic once the cycle-to-cycle changes of amplitude and period both stay below 10^{-3} for three consecutive cycles.

12.4.3 Grid independence: Richardson extrapolation and GCI

For a quantity f (St, $\bar{C}_D$) on fine, medium and coarse meshes (f_1, f_2, f_3) with representative cell sizes $h = \sqrt{\text{area/cells}}$ and ratios $r_{21} = h_2/h_1$, $r_{32} = h_3/h_2$ (Celik et al. 2008):

$$e_{21} = f_2 - f_1, \quad e_{32} = f_3 - f_2, \tag{12.1}$$

$$p = \frac{\left|\ln|e_{32}/e_{21}| + \ln\frac{r_{21}^p - s}{r_{32}^p - s}\right|}{\ln r_{21}}, \quad s = \operatorname{sign}(e_{32}/e_{21}) \quad \text{(solved by fixed-point iteration)}, \tag{12.2}$$

$$f_{ext} = \frac{r_{21}^p f_1 - f_2}{r_{21}^p - 1}, \qquad GCI_{fine} = \frac{1.25\,|e_{21}/f_1|}{r_{21}^p - 1}. \tag{12.3}$$

p is the observed order of accuracy (expected 1–2), f_{ext} the extrapolated grid-converged value, and GCI a conservative error band on the fine-mesh value (safety factor 1.25 for three-grid studies). The asymptotic ratio $GCI_{medium}/(r_{21}^p GCI_{fine}) \approx 1$ confirms the meshes are in the asymptotic range.

12.5 Results

12.5.1 The flow field

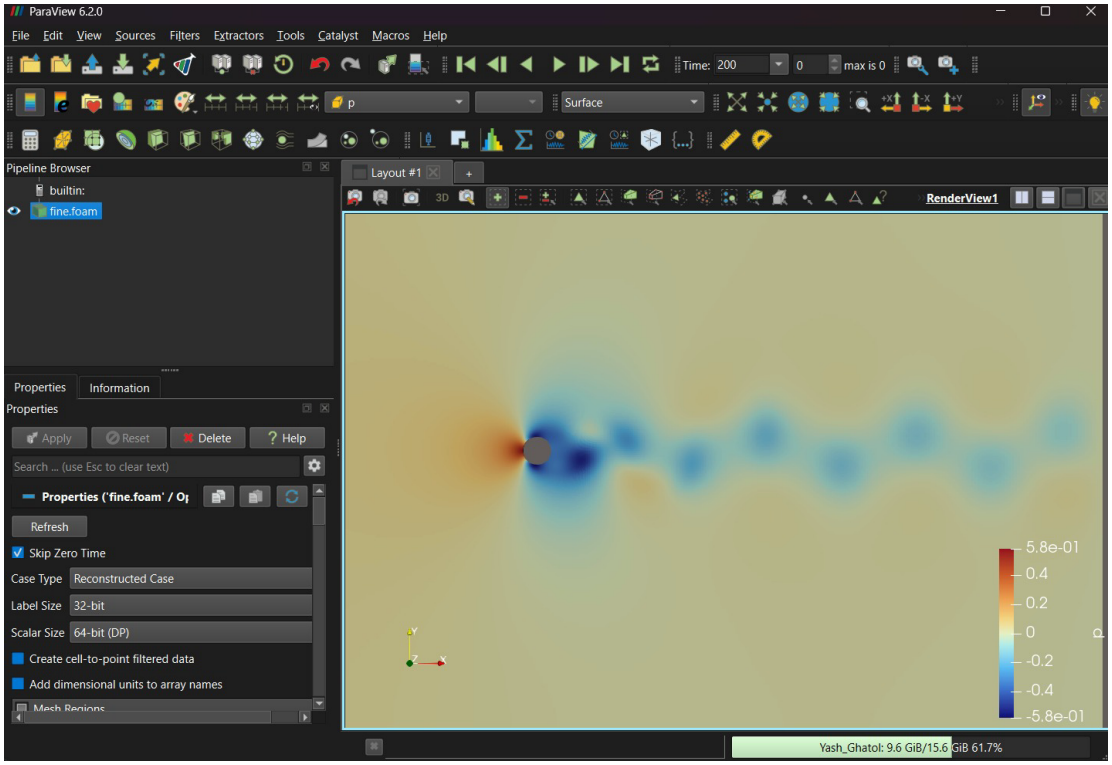


Figure 12.2: ParaView 6.2 screenshot (unedited) of the fine-mesh OpenFOAM solution at $t = 200$, coloured by pressure. High pressure (red) at the front stagnation point, low pressure on the sides, and a Kármán street of low-pressure vortex cores alternating above and below the centreline.

12.5.2 Grid independence

Mesh	Cells	St	$\bar{C}_D$	C_L amplitude
Coarse	6 690	0.1644	1.3754	0.3216
Medium	26 760	0.1688	1.3880	0.3451
Fine	**107 040**	**0.1695**	**1.3879**	**0.3451**
Richardson extrapolation	–	0.1697	converged	converged
2-D simulations, Re = 100 (approx.)	–	0.164–0.165	≈ 1.33	≈ 0.33

Table 12.2: Grid independence (analysis window: last 80 time units, 13–14 shedding periods per mesh; amplitude drift $\leq 0.04\,\%$).

Quantity	coarse→medium	medium→fine	p	f_{ext}	GCI_{medium}	GCI_{fine}	asympt. ratio
St	2.67 %	0.46 %	2.48	0.1697	0.71 %	0.13 %	1.005
$\bar{C}_D$	0.91 %	0.00 %	–	–	–	–	–
C_L amplitude	7.30 %	0.01 %	–	–	–	–	–

Table 12.3: Richardson extrapolation and grid convergence index, from `grid_independence.md`.

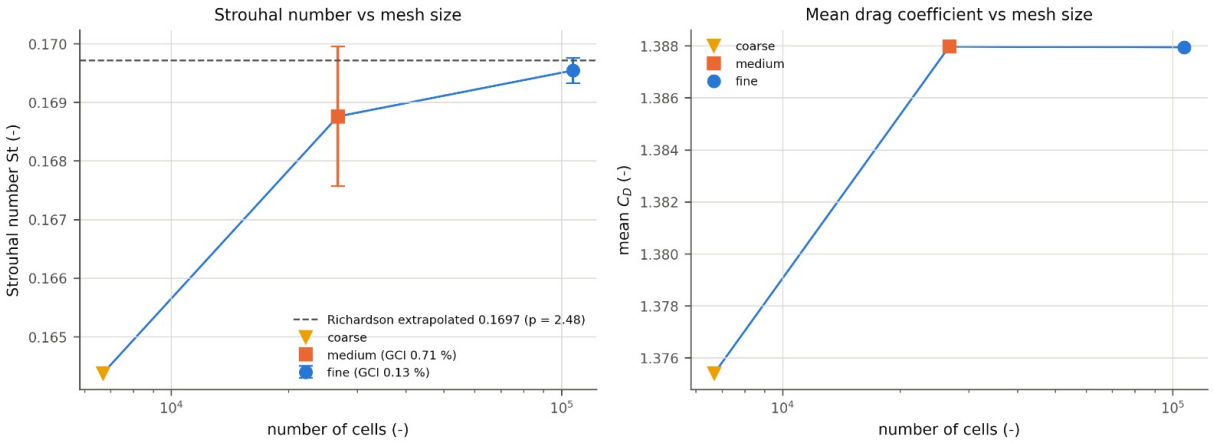


Figure 12.3: Strouhal number and mean drag vs number of cells; error bars on St are the GCI; dashed line the Richardson-extrapolated value.

- **St converges monotonically**: the change falls from 2.67 % to 0.46 % per refinement. The observed order $p = 2.48$ is close to the nominal second order of the schemes, the extrapolated value is 0.1697 and the fine-mesh GCI is 0.13 %. The asymptotic ratio 1.005 (≈ 1) shows the three meshes are in the asymptotic range.
- **Mean drag and lift amplitude converge already on the medium mesh** (changes 0.00 % and 0.01 %). With $e_{21} \approx 0$ the observed order is undefined (the ratio e_{32}/e_{21} blows up), so no Richardson value is reported: the medium–fine difference is below the resolution of the time averaging.
- The **coarse mesh** under-predicts both St and the lift amplitude (0.322 vs 0.345): with a 0.022 D first cell it under-resolves the boundary layer and the separation.

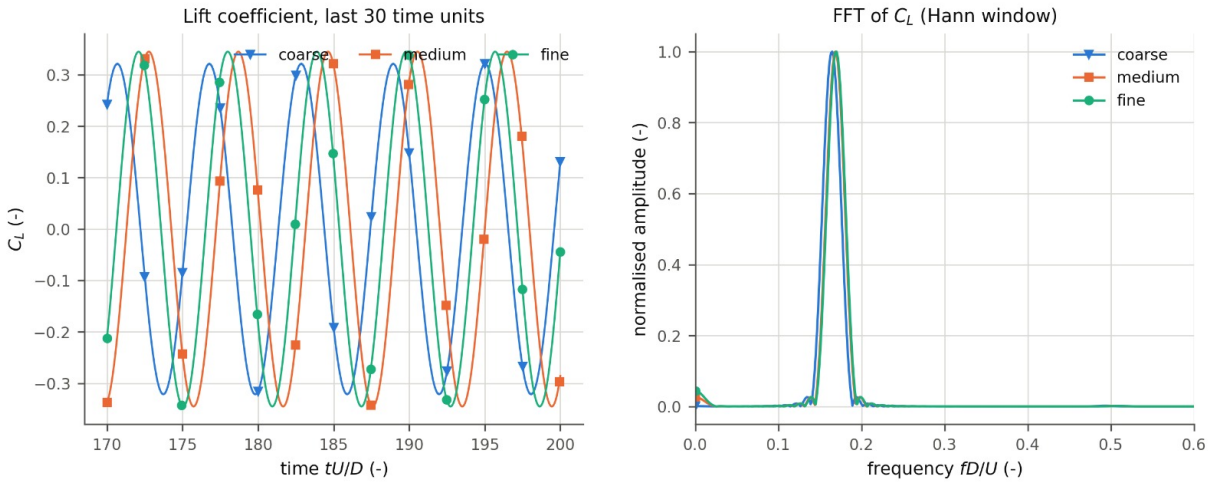


Figure 12.4: Lift coefficient over the last 30 time units for the three meshes, and its spectrum (Hann window). The single sharp peak at $fD/U \approx 0.17$ is the shedding frequency.

12.5.3 Residuals and periodicity

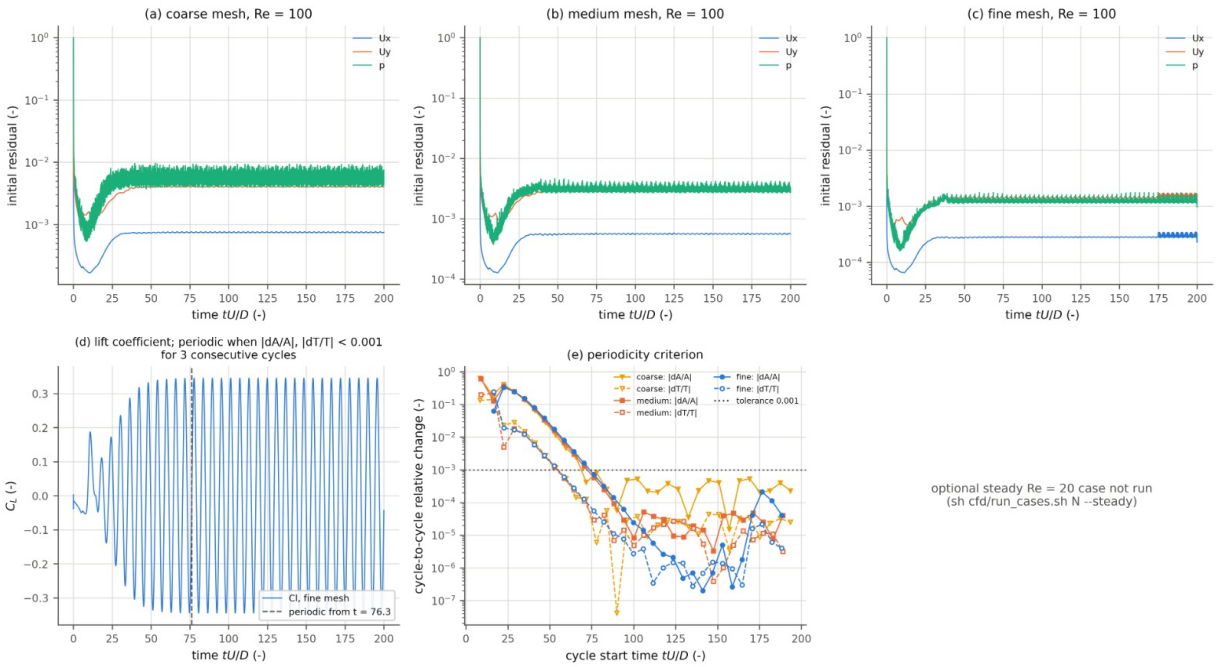


Figure 12.5: (a)–(c) Initial residuals of U_x, U_y and p for the three meshes; (d) fine-mesh lift, periodic from $t = 76.3$; (e) cycle-to-cycle relative change of the lift amplitude and period.

- **Residuals level off** (fine mesh: about 10^{-3} for p and U_y, 3×10^{-4} for U_x) instead of falling to zero. This is expected for a periodic flow: every time step starts from a changed flow (Section 12.4.2). Finer meshes level off lower. The small step on the fine mesh after $t = 175$ coincides with the start of window sampling, which also caps the time step to hit the sample times.
- **The wake is periodic from** $t = 76.3$ on the fine mesh: the cycle-to-cycle change of amplitude and period stays below 10^{-3}. The PIV window ($t = 175$–200) is therefore fully developed. The coarse mesh hovers around the tolerance, another sign of under-resolution.

12.5.4 Pressure reconstructed from the CFD velocity

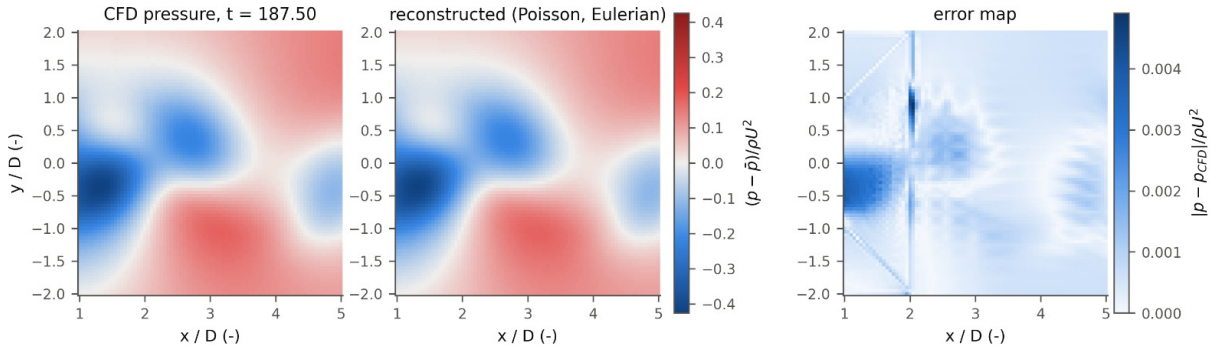


Figure 12.6: At $t = 187.5$: CFD pressure in the window, pressure reconstructed from the CFD velocity samples (Poisson, Eulerian, no noise, $h = 0.05D$), and the error map.

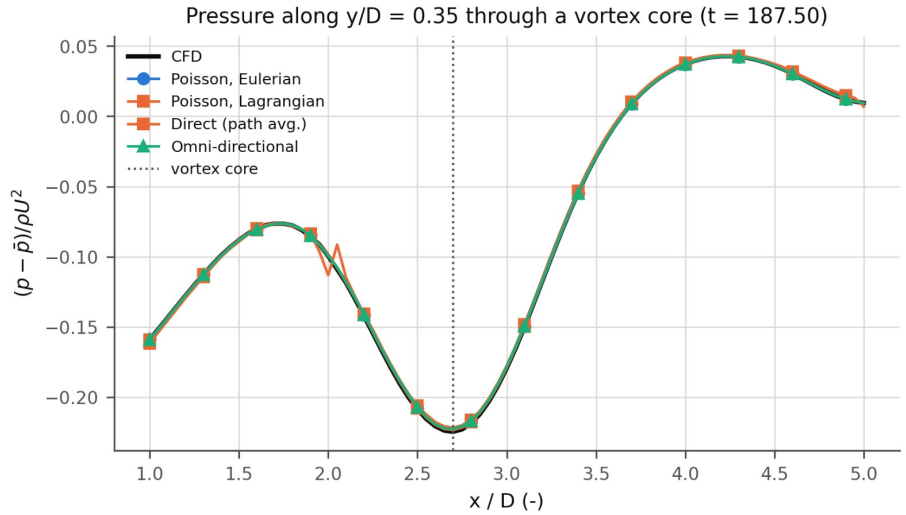


Figure 12.7: Pressure along $y/D = 0.35$, through a shed vortex core: CFD and four reconstructions.

1. **The wake pressure is recovered**: without noise the Poisson error is $\varepsilon_{L2} = 0.5$–$0.75\,\%$ over a shedding period. All methods reproduce the vortex-core suction ($-0.22\rho U^2$ at $x/D \approx 2.7$) within about 1 %.

2. **The error lies on the CFD mesh's block boundaries**: the vertical line $x/D = 2$ and the diagonals towards $(2, \pm 2)$ are exactly the edges of the O-grid around the cylinder, which extends to the square $[-2, 2]^2$. The sampled velocity (interpolated from cell values) has small slope discontinuities across these block edges, where cell size and orientation change; differentiation turns these kinks into steps in $\mathbf{G}$. Direct integration, which carries every error along its path, shows a visible kink at $x/D = 2$. This is a systematic error of the "measured" data, analogous to a calibration error in real PIV, and it is amplified by exactly the mechanism of Chapter 4.

3. **The largest error is near the inflow edge of the window** ($x/D \approx 1$–1.7, $y < 0$), where a new vortex is forming close to the cylinder and gradients are steepest.

12.5.5 Time step, scheme and noise on the wake

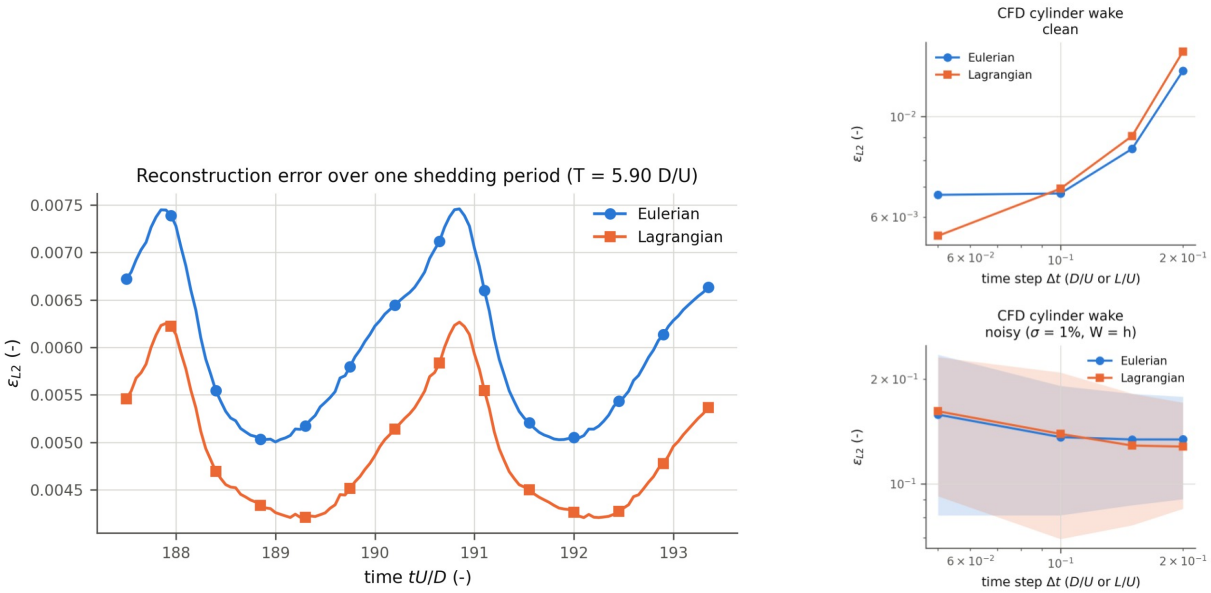


Figure 12.8: Left: reconstruction error over one shedding period ($T = 5.90$, clean, $\Delta t = 0.05$). Right: error vs time step on the wake, clean (top) and with $\sigma = 1\,\%$ noise (bottom).

1. **The error pulses twice per shedding period** ($T = 5.90$, so $St = 1/T = 0.1695$, consistent with the FFT). Each peak occurs as a new vortex forms on one side of the cylinder, near the inflow edge of the window.
2. **At $\Delta t = 0.05$ the Lagrangian scheme is 15–18 % better** throughout the period ($\varepsilon \approx 0.0042$–$0.0063$ vs 0.0050–0.0075), as predicted for structures carried through the window.
3. **At $\Delta t \geq 0.1$ the Eulerian scheme is slightly better** (e.g. 8.4×10^{-3} vs 9.1×10^{-3} at $\Delta t = 0.15$), unlike the convected Taylor–Green vortex. The theory of Section 3.2 assumes a pattern simply carried along ($w \ll c$). In the near wake ($x/D = 1$–5) the vortices are still forming: they move slower than U_∞ and rotate quickly (w comparable to c), so particle paths curve strongly and the straight-line tracking error grows quickly with Δt. More tracked points also leave the $4D \times 4D$ window at larger Δt and fall back to the Eulerian estimate. Both are plausible contributors; separating them would need the fallback fraction and a longer window.
4. **With 1 % noise** the error is $\varepsilon \approx 0.13$–0.15 for both schemes and almost flat in Δt: noise amplification dominates truncation. It is larger than on Taylor–Green (0.08) because the wake's pressure variations are only about $\pm 0.2 \rho U^2$, so the same velocity noise is larger relative to the signal.

What the CFD case adds

The analytic tests isolate each error source; the wake combines them on a realistic unsteady flow. The reconstruction recovers the wake pressure to better than 1 % from clean data, the Lagrangian advantage holds only at small Δt in the vortex-formation region, and systematic kinks in the data (here from the CFD mesh) are amplified just like random noise.

Chapter 13

Comparison with published studies

The project's controlled synthetic experiments can be checked against the conclusions of the PIV-pressure literature.

This project	Published finding	Match
Exact p on 2 edges: ε 0.044 vs 0.082 all-Neumann ($\approx 2\times$)	Pan et al. (2016): Dirichlet boundaries give less error than Neumann (about $2\times$ for large domains with boundary error); avoid pure Neumann where possible	Agrees
Bernoulli edges: ε 0.30–0.81 in rotational flow	Pan et al. (2016): Bernoulli-based Dirichlet data are valid only in irrotational regions	Agrees
Errors from 0.002 % (clean, fine) to > 100 % (direct, $\sigma \geq 2$ %); depend on method, spacing, filtering	Charonko et al. (2010), as summarised by Pan et al.: errors from < 1 % to > 100 %, sensitive to all these factors, no single best method	Agrees
Lagrangian $3\times$ better at large Δt in convected flow; noise $\propto 1/\Delta t$	Violato et al. (2011): the Eulerian approach is limited to short Δt; short Δt amplifies precision error, long Δt adds truncation error	Agrees
Omni and Poisson recover the main features; direct integration very noise-sensitive	van Gent et al. (2017): all methods tested recover the main features; noise lowers accuracy but no method was critically sensitive at their noise levels	Partly
Cylinder wake (fine mesh): $St = 0.1695$, $\bar{C}_D = 1.388$, C_L amplitude 0.345	2-D simulations at Re = 100 (e.g. Park et al. 1998): $St \approx 0.164$–0.165, $\bar{C}_D \approx 1.33$; experiments $St \approx 0.164$ (Williamson 1996)	Agrees

Why "partly" for van Gent et al. Their comparison used tomographic PIV and Lagrangian particle tracking data with more advanced methods (least-squares pseudo-tracking, vortex-in-cell, FlowFit); simple single-path direct integration was not among them. Our direct-integration result does not contradict them, it covers a method they did not test.

Why the wake values are 3–4 % high. The $\pm 10D$ symmetry sides block 5 % of the channel. The flow accelerates past the cylinder, which raises the drag and the shedding frequency; a few per cent is a plausible size for 5 % blockage, though we did not run a wider domain to confirm it. The literature values are for (nearly) unbounded domains and are quoted approximately; confirm them in the papers before quoting exact numbers.

What the literature adds that we did not test. Pan et al. show that the error also depends on the domain size and shape (for Dirichlet domains of fixed area, a square is the worst case). Our windows are all square, so this is a natural next study.

Chapter 14

Conclusions

14.1 Key findings

1. **The code is verified.** All three algorithms are second-order accurate on exact Navier–Stokes solutions (slopes 1.8–2.1), including the boundaries.
2. **Noise, not discretisation, limits PIV pressure.** The error grows linearly with the velocity noise, and there is an optimum vector spacing ($h \approx 0.03$ here) below which noise amplification by the derivatives dominates.
3. **The integration method matters.** Poisson and omni-directional integration are 5–7$\times$ more robust than direct integration because they average errors over many paths; Poisson is about 27$\times$ faster than omni.
4. **Boundary data matter.** Known pressure on two opposite edges halves the error; Bernoulli edges are valid only in irrotational flow; with pure Neumann data, a zero-mean constraint beats pinning a node.
5. **The time step is a trade-off.** The Lagrangian material derivative tolerates larger Δt in convected flow; with noisy data a larger Δt helps both schemes.
6. **Smoothing trades noise for bias.** Denoising cuts the noisy error about 3$\times$ but hurts clean data.
7. **The CFD wake.** The OpenFOAM solution is grid-independent (St = 0.1695, GCI 0.13 %) and periodic from $t = 76$. From its velocity the pressure is reconstructed to 0.5–0.75 %; the remaining error sits on the CFD mesh's block edges, where kinks in the data are amplified like noise. The Lagrangian advantage holds only at small Δt in the near wake.
8. **Solvers.** SOR (at the Neumann-corrected ω_{opt}) and CG need 80–140$\times$ fewer iterations than Jacobi; classic Jacobi stalls on the all-Neumann problem; a small change per iteration is not proof of convergence.

14.2 Objectives

All four objectives are met: the algorithms are implemented, verified on exact flows, studied parametrically, and compared with a CFD simulation and with published studies.

Main takeaway

PIV pressure is limited by how velocity noise is amplified by differentiation. The choice of method, boundary data, vector spacing and time separation decides whether that noise is averaged out or accumulated.

14.3 Recommended practice

Use a Poisson solver; impose Dirichlet values only where pressure is genuinely known (taps, free stream); use the Lagrangian material derivative with the largest Δt that keeps truncation

error small; choose the vector spacing near the noise–truncation optimum rather than the finest possible; apply light denoising above $\sigma \approx 0.5\,\%$.

14.4 Limitations and future work

- **Noise model:** σ is independent of the window size and uncorrelated between neighbouring vectors. Real PIV has less random error for larger windows and spatially correlated errors from overlapping windows.
- **Two-dimensional data:** planar PIV of 3-D turbulence misses $-\rho w\,\partial \mathbf{u}/\partial z + \mu\,\partial^2 \mathbf{u}/\partial z^2$, an $O(1)$ error that refinement cannot remove. The project includes a loader and out-of-plane term calculator for turbulence-database slices (JHTDB) but no data were analysed.
- **Omni-directional method:** a rotating-ray variant of Liu & Katz (2006), not a line-by-line copy.
- **Domain shape:** only square windows were tested.
- **CFD window:** the $4D \times 4D$ window sits in the vortex-formation region and crosses mesh block edges; a window further downstream on a uniform-mesh region would separate the two effects.
- **Next steps:** window-dependent noise; correlated noise; a downstream CFD window; 3-D time-resolved data.

Chapter A

Viva preparation: questions and short answers

The questions below collect the ones already given in the chapters plus those an examiner is likely to ask about the whole project.

Big picture

Why is pressure from PIV an ill-posed / sensitive problem?
The data must be differentiated before being integrated, and differentiation amplifies noise as $1/h$, $1/h^2$ and $1/\Delta t$.

What is the single most important result?
There is an optimum vector spacing: finer PIV grids can give worse pressure.

Which method would you use in practice, and why?
Poisson: as robust as omni (both average errors), 27× faster, and well understood (least-squares projection).

Why use synthetic data instead of a real experiment?
The true pressure must be known everywhere to measure the error. Exact solutions and CFD give it; one can also control each error source separately.

Physics

Derive the pressure gradient from Navier–Stokes.
Rearrange momentum: $\nabla p = -\rho D\mathbf{u}/Dt + \mu\nabla^2\mathbf{u}$.

Why is Taylor–Green an exact solution?
$\nabla^2\mathbf{u} = -2\mathbf{u}$, so the unsteady and viscous terms cancel with $F = e^{-2\nu t}$, and the convective term is a gradient balanced by pressure.

Why is the convected Taylor–Green exact?
Galilean invariance: $\mathbf{u}' = \mathbf{c} + \mathbf{u}(\mathbf{x} - \mathbf{c}t)$ has the same material derivative.

What sets λ in Kovasznay flow?
Substituting the ansatz gives $\lambda^2 - Re\,\lambda - 4\pi^2 = 0$; the negative root gives a decaying wake.

When does the Lagrangian derivative beat the Eulerian one?
When structures are convected faster than their own velocity ($w < c$): errors scale as w^3 vs c^3.

Why does Bernoulli fail here?
Both flows are rotational everywhere; Bernoulli holds across the field only in irrotational flow.

Numerics

Derive the one-sided first-derivative stencil.
Taylor-expand f_1, f_2; cancel f_0 and f''; get $(-3f_0 + 4f_1 - f_2)/2h$.

How are Neumann conditions imposed?
Ghost points eliminated with a central difference of the condition; corners use both conditions.

Why is the all-Neumann system singular, and how do you fix it?
$A\mathbf{1} = 0$; pin a node or add a zero-mean constraint (default).

What is the compatibility residual?
$\iint \nabla \cdot \mathbf{G} - \oint \mathbf{n} \cdot \mathbf{G}$; non-zero when $\mathbf{G}$ is not an exact gradient.

Why does Poisson filter noise?
It is the least-squares projection of $\mathbf{G}$ onto gradient fields; the curl is discarded.

Why is direct integration path dependent?
Measured $\mathbf{G}$ has a curl; errors along a path add like a random walk.

Why is the velocity-source Poisson form worse with noise?
Its source and its boundary data no longer come from one vector field, and it multiplies noisy derivatives.

Why does an $O(h)$ boundary row still give $O(h^2)$ global error?
The error acts on an $O(h)$ strip; the scaled scheme is a second-order finite-volume scheme.

Solvers and convergence

Show Jacobi is pseudo-time marching.
$p^{k+1} = p^k + \Delta\tau(\nabla_h^2 p^k - f)$, $\Delta\tau = \omega h^2/4$.

Why does classic Jacobi stall?
The Neumann checkerboard mode has amplification -1.

What is ω_{opt} for SOR here?
$2/(1+\sqrt{1-\rho_J^2})$ with $\rho_J = \cos^2(\pi h/2L)$ for Neumann walls: 1.932 at 64^2.

Why does SOR diverge for $\omega \geq 2$?
Kahan: $\rho_{SOR} \geq |\omega - 1|$.

Is a small change per iteration enough to stop?
No; check the residual too. With slow convergence the true error is $\Delta/(1-\rho)$.

Why normalise norms?
So the same physical imbalance gives the same number on any grid.

CFD

Why is a 2-D simulation valid at Re = 100?
The wake stays two-dimensional below Re $\approx$ 190.

Why an O-grid?
Body-fitted, near-orthogonal cells, clustered at the wall for the boundary layer.

Why the inlet kick?
A symmetric set-up delays shedding; the kick breaks symmetry and is removed at $t = 6$.

How is St computed?
FFT of $C_L(t)$ after uniform resampling and a Hann window; $St = fD/U$.

Why might St and C_D be higher than literature?
5 % blockage from the $\pm 10D$ symmetry sides.

What does GCI = 0.5 % mean?
The fine-mesh value lies within about 0.5 % of the grid-converged value (safety factor 1.25).

Why don't residuals fall to zero?
The flow is periodic; each step starts from a changed flow. Use a periodicity check instead.

How does PISO relate to this project?
It solves a pressure Poisson equation from velocity every time step: the same mathematical step as the reconstruction.

Chapter B

File map and how to run

B.1 Files

File	Role
`fields/field.py`	`FlowField`, the common data interface
`fields/analytic.py`	Taylor–Green (incl. convected) and Kovasznay solutions
`fields/cfd_loader.py`, `jhtdb_loader.py`	CFD window and turbulence-database loaders
`piv/degrade.py`, `denoise.py`	synthetic PIV model; Gaussian and Savitzky–Golay filters
`core/derivatives.py`	finite-difference stencils (order 1 or 2)
`core/material_derivative.py`	Eulerian/Lagrangian $D\mathbf{u}/Dt$ and $\mathbf{G}$
`core/poisson.py`	matrix assembly, ghost points, compatibility, direct/CG solve
`core/iterative.py`	Jacobi, Gauss–Seidel, SOR, CG with residual histories
`core/direct_integration.py`, `omni_integration.py`	the other two integration methods
`core/pipeline.py`	`reconstruct()`, the single entry point
`analysis/metrics.py`, `norms.py`	error metrics and residual norms
`analysis/plots.py`, `summary.py`	shared plot style; `SUMMARY.md` generator
`studies/run_*.py`	one script per parameter sweep
`cfd/generate_cases.py`	writes the OpenFOAM cases and `mesh_info.json`
`cfd/cases/*/Allrun`	mesh and run one case (serial or parallel)
`cfd/export_window.py`	window samples → `cylinder_re100_window.npz`
`cfd/grid_independence.py`	St, C_D, C_L, Richardson/GCI
`cfd/parse_log.py`, `change_norm.py`	solver residuals; periodicity check
`tests/`	87 unit tests
`run_all.py`	runs every study and the summary

B.2 Commands

Analytic studies (any OS, from `piv_pressure/`):

```
python -m pip install -r requirements.txt
python -m pytest -q            # 87 passed
python run_all.py --quick      # ~40 s check
python run_all.py              # full run, ~8 min
```

CFD on native Windows with the OpenFOAM MinGW build (OpenFOAM terminal for the runs, PowerShell for Python):

```
python cfd/generate_cases.py                                # PowerShell
sed -i 's/\r$//' cfd/cases/*/Allrun                         # OpenFOAM terminal
sed -i 's/mpirun -np/mpiexec -n/' cfd/cases/*/Allrun        # MS-MPI only has mpiexec
sh cfd/cases/coarse/Allrun 4
sh cfd/cases/medium/Allrun 4
sh cfd/cases/fine/Allrun 4
python cfd/grid_independence.py                             # PowerShell, after the runs
python cfd/export_window.py
python cfd/parse_log.py
python cfd/change_norm.py
python run_all.py
```

Chapter C

Notation

Symbol	Meaning
$\mathbf{u} = (u, v)$	velocity
p, p_{ref}	reconstructed and true pressure (kinematic in CFD, $\rho = 1$)
ρ, μ, $\nu = \mu/\rho$	density, dynamic and kinematic viscosity
$\mathbf{G}$	pressure gradient estimated from velocity, $-\rho D\mathbf{u}/Dt + \mu\nabla^2\mathbf{u}$
h ($\Delta x, \Delta y$)	PIV vector spacing
W	interrogation-window width
σ	velocity noise level, fraction of U
Δt	time between PIV snapshots
ε_{L2}, ε_{max}	relative L_2 error; max error over $\frac{1}{2}\rho U^2$
R	compatibility residual
A, b, w	Poisson matrix, right-hand side, trapezoidal weights
ω	relaxation factor (Jacobi, SOR)
ρ_J, ρ_{SOR}	spectral radii of the iteration matrices
St, C_D, C_L	Strouhal number, drag and lift coefficients
p (Richardson), GCI	observed order; grid convergence index

References

[1] B. W. van Oudheusden, PIV-based pressure measurement, *Meas. Sci. Technol.* 24 (2013) 032001.

[2] J. J. Charonko, C. V. King, B. L. Smith, P. P. Vlachos, Assessment of pressure field calculations from particle image velocimetry measurements, *Meas. Sci. Technol.* 21 (2010) 105401.

[3] Z. Pan, J. Whitehead, S. Thomson, T. Truscott, Error propagation dynamics of PIV-based pressure field calculations: how well does the pressure Poisson solver perform inherently?, *Meas. Sci. Technol.* 27 (2016) 084012. `https://arxiv.org/abs/1602.00037`

[4] P. L. van Gent et al., Comparative assessment of pressure field reconstructions from particle image velocimetry measurements and Lagrangian particle tracking, *Exp. Fluids* 58 (2017).

[5] X. Liu, J. Katz, Instantaneous pressure and material acceleration measurements using a four-exposure PIV system, *Exp. Fluids* 41 (2006) 227–240.

[6] D. Violato, P. Moore, F. Scarano, Lagrangian and Eulerian pressure field evaluation of rod–airfoil flow from time-resolved tomographic PIV, *Exp. Fluids* 50 (2011) 1057–1070.

[7] R. de Kat, B. W. van Oudheusden, Instantaneous planar pressure determination from PIV in turbulent flow, *Exp. Fluids* 52 (2012).

[8] J. Park, K. Kwon, H. Choi, Numerical solutions of flow past a circular cylinder at Reynolds numbers up to 160, *KSME Int. J.* 12 (1998) 1200–1205.

[9] C. H. K. Williamson, Vortex dynamics in the cylinder wake, *Annu. Rev. Fluid Mech.* 28 (1996) 477–539.

[10] L. I. G. Kovasznay, Laminar flow behind a two-dimensional grid, *Proc. Camb. Philos. Soc.* 44 (1948) 58–62.

[11] I. B. Celik et al., Procedure for estimation and reporting of uncertainty due to discretization in CFD applications, *J. Fluids Eng.* 130 (2008) 078001.

[12] D. M. Young, *Iterative Solution of Large Linear Systems*, Academic Press, 1971.

Notes. The findings attributed to [3], [4] and [6] were checked against the papers' abstracts or text; [2] as summarised in [3]. Literature values for the cylinder wake ([8]) are approximate and should be confirmed from the paper before quoting.